```
#register num:241801041
#name:Darshan SP
from google.colab import files

uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
#register num:241801041
#name:Darshan SP
import pandas as pd

df = pd.read_csv("train.csv")

print(df.head())
```

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

```
#register num:241801041
#name:Darshan SP
print(df.tail())
```

```
     PassengerId  Survived  Pclass                                      Name  \
886          887         0       2                     Montvila, Rev. Juozas
887          888         1       1              Graham, Miss. Margaret Edith
888          889         0       3  Johnston, Miss. Catherine Helen "Carrie"
889          890         1       1                     Behr, Mr. Karl Howell
890          891         0       3                       Dooley, Mr. Patrick

        Sex   Age  SibSp  Parch      Ticket   Fare Cabin Embarked
886    male  27.0      0      0      211536  13.00   NaN        S
887  female  19.0      0      0      112053  30.00   B42        S
888  female   NaN      1      2  W./C. 6607  23.45   NaN        S
889    male  26.0      0      0      111369  30.00  C148        C
890    male  32.0      0      0      370376   7.75   NaN        Q
```

```
#register num:241801041
#name:Darshan SP
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
```

```
memory usage: 83.7+ KB
None
```

```
#register num:241801041
#name:Darshan SP
print(df.shape)
```

```
(891, 12)
```

```
#register num:241801041
#name:Darshan SP
print(df.columns)
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

```
#register num:241801041
#name:Darshan SP
print(df.isnull().sum())
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

```
#register num:241801041
#name:Darshan SP
df["Age"] = df["Age"].ffill()
```

```
#register num:241801041
#name:Darshan SP
df["Age"] = df["Age"].bfill()
```

```
#register num:241801041
#name:Darshan SP
df = df.drop_duplicates()
```

```
#register num:241801041
#name:Darshan SP
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df["Sex"] = le.fit_transform(df["Sex"])
```

```
#register num:241801041
#name:Darshan SP
print(df.head())
```

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name  Sex   Age  SibSp  Parch  \
0                            Braund, Mr. Owen Harris    1  22.0      1      0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...    0  38.0      1      0
2                             Heikkinen, Miss. Laina    0  26.0      0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)    0  35.0      1      0
4                           Allen, Mr. William Henry    1  35.0      0      0

             Ticket     Fare Cabin Embarked
0         A/5 21171   7.2500   NaN        S
```

```
1          PC 17599  71.2833   C85        C
2  STON/O2. 3101282   7.9250   NaN        S
3            113803  53.1000  C123        S
4            373450   8.0500   NaN        S
```

```
#register num:241801041
#name:Darshan SP
print(df.isnull().sum())
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

```
#register num:241801041
#name:Darshan SP
df["Cabin"] = df["Cabin"].fillna("Unknown", limit=5)
```

```
print(df["Cabin"].head(10))
```

```
0    Unknown
1        C85
2    Unknown
3       C123
4    Unknown
5    Unknown
6        E46
7    Unknown
8        NaN
9        NaN
Name: Cabin, dtype: object
```

```
#register num:241801041
#name:Darshan SP
df = df.drop_duplicates()
```

```
print(df.duplicated().sum())
```

```
0
```

```
#register num:241801041
#name:Darshan SP
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

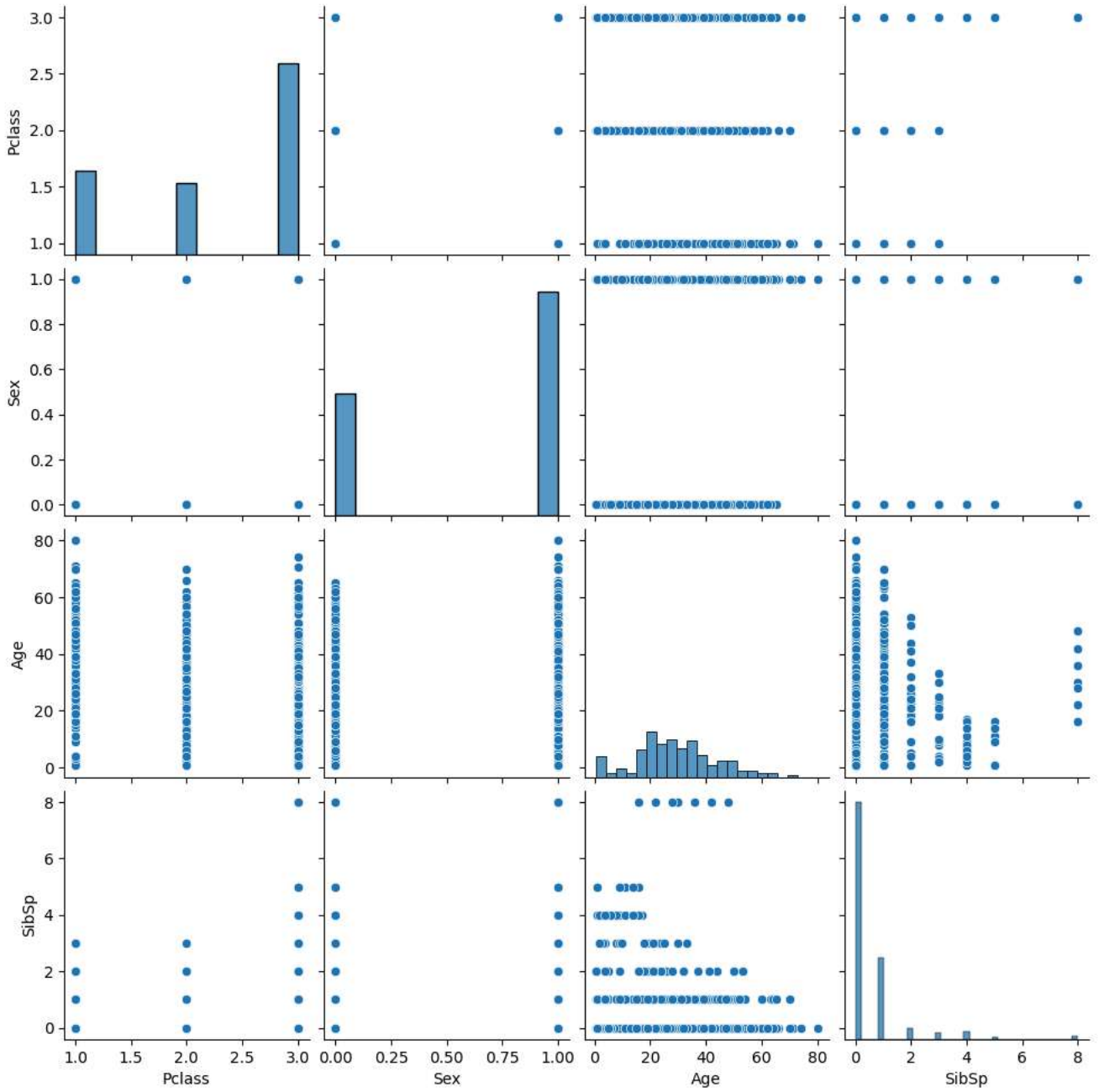
df["Fare"] = scaler.fit_transform(df[["Fare"]])
```

```
print(df["Fare"].head())
```

```
0   -0.502445
1    0.786845
2   -0.488854
3    0.420730
4   -0.486337
Name: Fare, dtype: float64
```

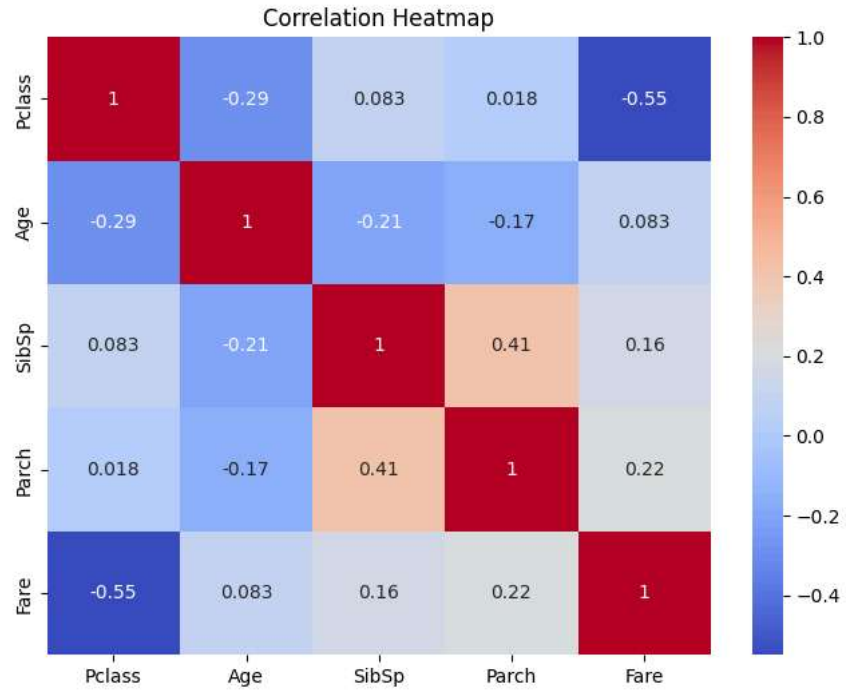
```
#register num:241801041
#name:Darshan SP
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(df[["Pclass", "Sex", "Age", "SibSp"]])
plt.show()
```




```
#register num:241801041
#name:Darshan SP
corr = df[["Pclass", "Age", "SibSp", "Parch", "Fare"]].corr()

plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```


Correlation Heatmap
Pclass
Age
SibSp
Parch
Fare
1
-0.29
0.083
0.018
-0.55
-0.29
1
-0.21
-0.17
0.083
0.083
-0.21
1
0.41
0.16
0.018
-0.17
0.41
1
0.22
-0.55
0.083
0.16
0.22
1
Pclass
Age
SibSp
Parch
Fare
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

EXP-01 1b(PODS)

```
#register num:241801041
#name:Darshan SP
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
#register num:241801041
#name:Darshan SP
import pandas as pd

url = "https://raw.githubusercontent.com/Giskard-AI/examples/main/datasets/WA_Fn-UseC_-Telco-Customer-Churn.csv"

df = pd.read_csv(url)

print("Dataset loaded successfully!")
print("Shape:", df.shape)

display(df.head())
```

```
Dataset loaded successfully!
Shape: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No

5 rows × 21 columns

```
#register num:241801041
#name:Darshan SP
print("First 10 Records:")
display(df.head(10))
print("Last 10 Records:")
display(df.tail(10))
print("Shape of Dataset:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
```

```
First 10 Records:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	Nc
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No
5	9305-CDSKC	Female	0	No	No	8	Yes	Yes	Fiber optic	No
6	1452-KIOVK	Male	0	No	Yes	22	Yes	Yes	Fiber optic	No
7	6713-OKOMC	Female	0	No	No	10	No	No phone service	DSL	Yes
8	7892-POOKP	Female	0	Yes	No	28	Yes	Yes	Fiber optic	No
9	6388-TABGU	Male	0	No	Yes	62	Yes	No	DSL	Yes

10 rows × 21 columns

```
Last 10 Records:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurit
7033	9767-FFLEM	Male	0	No	No	38	Yes	No	Fiber optic	N
7034	0639-TSIQW	Female	0	No	No	67	Yes	Yes	Fiber optic	Ye
7035	8456-QDAVC	Male	0	No	No	19	Yes	No	Fiber optic	N
7036	7750-EYXWZ	Female	0	No	No	12	No	No phone service	DSL	N
7037	2569-WGERO	Female	0	No	No	72	Yes	No	No	No intern servic
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	Ye
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	N
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	Ye
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	N
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	Ye

10 rows × 21 columns

```
Shape of Dataset:
(7043, 21)

Column Names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
```

```
#register num:241801041
#name:Darshan SP
print("Dataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
```

```
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
#register num:241801041
#name:Darshan SP
print("Summary Statistics:")
display(df.describe())
```

```
Summary Statistics:
```

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

```
#register num:241801041
#name:Darshan SP
print("Missing Values in Each Column:")
print(df.isnull().sum())
```

```
Missing Values in Each Column:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
#register num:241801041
#name:Darshan SP
print("Number of Duplicate Records:")
print(df.duplicated().sum())
```

```
Number of Duplicate Records:
0
```

```
#register num:241801041
#name:Darshan SP
print("Shape before removing duplicates:", df.shape)
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)
```

```
Shape before removing duplicates: (7043, 21)
Shape after removing duplicates: (7043, 21)
```

```
#register num:241801041
#name:Darshan SP
print(df.dtypes)
```

```
customerID           object
gender               object
SeniorCitizen         int64
Partner              object
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object
dtype: object
```

```
#register num:241801041
#name:Darshan SP
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
```

```
#register num:241801041
#name:Darshan SP
print("Missing values after converting TotalCharges:")
print(df.isnull().sum())
```

```
Missing values after converting TotalCharges:
customerID           0
gender               0
SeniorCitizen        0
Partner              0
Dependents           0
tenure               0
PhoneService         0
MultipleLines        0
InternetService      0
OnlineSecurity       0
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
MonthlyCharges       0
TotalCharges        11
Churn                0
dtype: int64
```

```
#register num:241801041
#name:Darshan SP
```

```
numerical_columns = df.select_dtypes(include=np.number).columns
for column in numerical_columns:
    df[column] = df[column].fillna(df[column].median())
```

```
#register num:241801041
#name:Darshan SP
categorical_columns = df.select_dtypes(include="object").columns
for column in categorical_columns:
    df[column] = df[column].fillna(df[column].mode()[0])
```

```
#register num:241801041
#name:Darshan SP
print("Missing Values After Cleaning:")
print(df.isnull().sum())
```

```
Missing Values After Cleaning:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
#register num:241801041
#name:Darshan SP
print("Total Missing Values:")
print(df.isnull().sum().sum())
```

```
Total Missing Values:
0
```

```
#register num:241801041
#name:Darshan SP
categorical_columns = [
    "gender",
    "Partner",
    "Dependents",
    "Contract",
    "PaymentMethod",
    "InternetService",
    "Churn"
]
label_encoder = LabelEncoder()
for column in categorical_columns:
    df[column] = label_encoder.fit_transform(df[column])
display(df.head())
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	0	0	1	0	1	No	No phone service	0	No
1	5575-GNVDE	1	0	0	0	34	Yes	No	0	Yes
2	3668-QPYBK	1	0	0	0	2	Yes	No	0	Yes
3	7795-CFOCW	1	0	0	0	45	No	No phone service	0	Yes
4	9237-HQITU	0	0	0	0	2	Yes	No	1	No

5 rows × 21 columns

```
#register num:241801041
#name:Darshan SP
print(df.dtypes)
```

```
customerID           object
gender                int64
SeniorCitizen         int64
Partner               int64
Dependents            int64
tenure                int64
PhoneService         object
MultipleLines        object
InternetService       int64
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract              int64
PaperlessBilling     object
PaymentMethod         int64
MonthlyCharges      float64
TotalCharges        float64
Churn                 int64
dtype: object
```

```
#register num:241801041
#name:Darshan SP

features_to_scale = [
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]
scaler = StandardScaler()
df[features_to_scale] = scaler.fit_transform(df[features_to_scale])
display(df[features_to_scale].head())
```

	tenure	MonthlyCharges	TotalCharges
0	-1.277445	-1.160323	-0.994242
1	0.066327	-0.259629	-0.173244
2	-1.236724	-0.362660	-0.959674
3	0.514251	-0.746535	-0.194766
4	-1.236724	0.197365	-0.940470

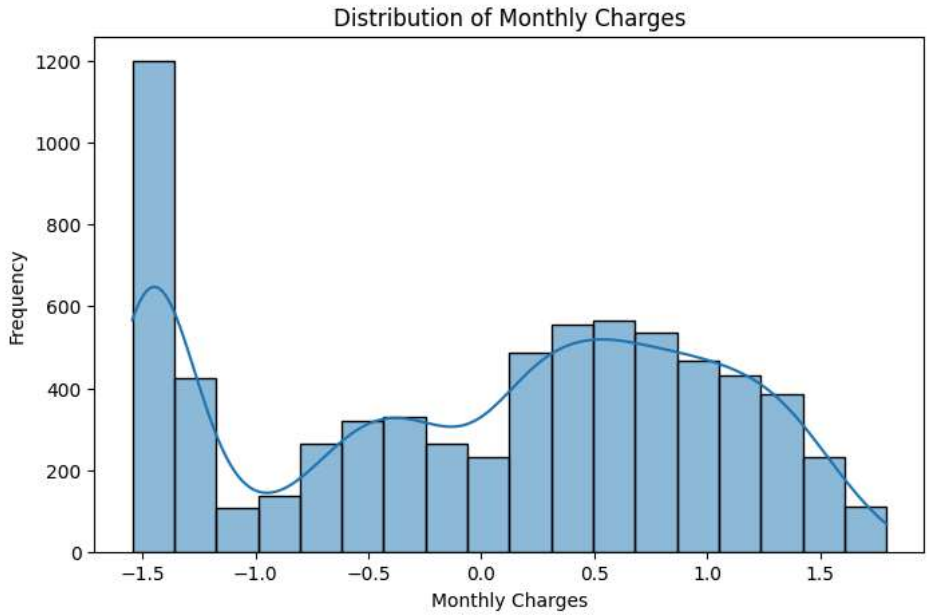
```
#register num:241801041
#name:Darshan SP
print("Mean after standardization:")
print(df[features_to_scale].mean())
print("\nStandard deviation after standardization:")
print(df[features_to_scale].std())
```

```
Mean after standardization:
tenure           -2.421273e-17
```

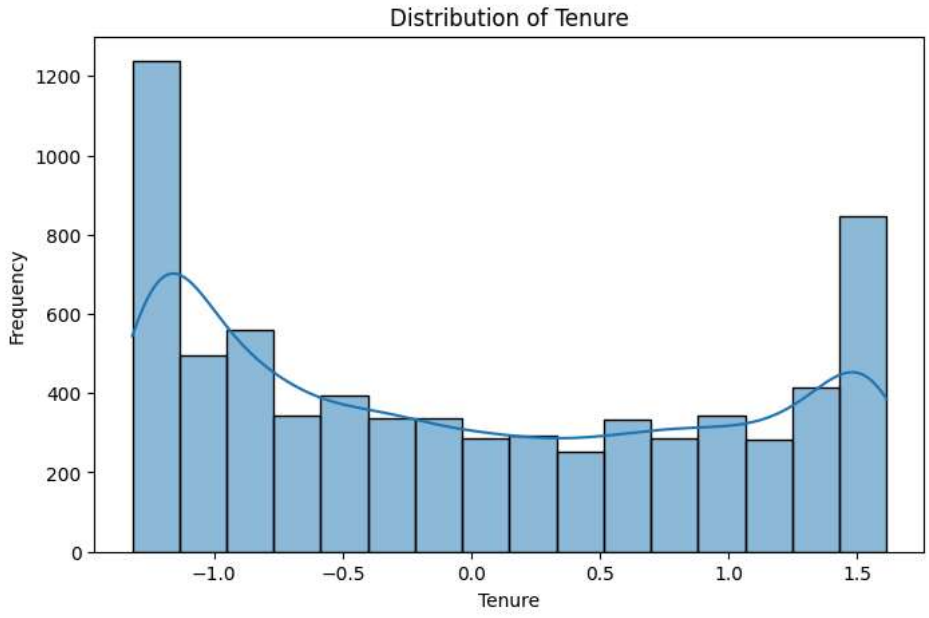
```
MonthlyCharges   -6.406285e-17
TotalCharges     -1.488074e-17
dtype: float64

Standard deviation after standardization:
tenure            1.000071
MonthlyCharges    1.000071
TotalCharges      1.000071
dtype: float64
```

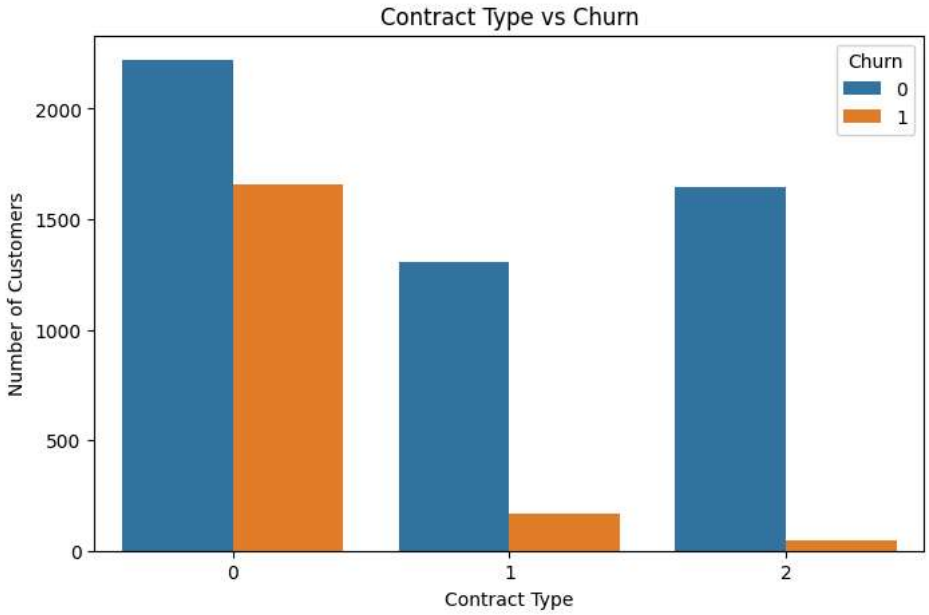
```
#register num:241801041
#name:Darshan SP
plt.figure(figsize=(8, 5))
sns.histplot(df["MonthlyCharges"], kde=True)
plt.title("Distribution of Monthly Charges")
plt.xlabel("Monthly Charges")
plt.ylabel("Frequency")
plt.show()
```



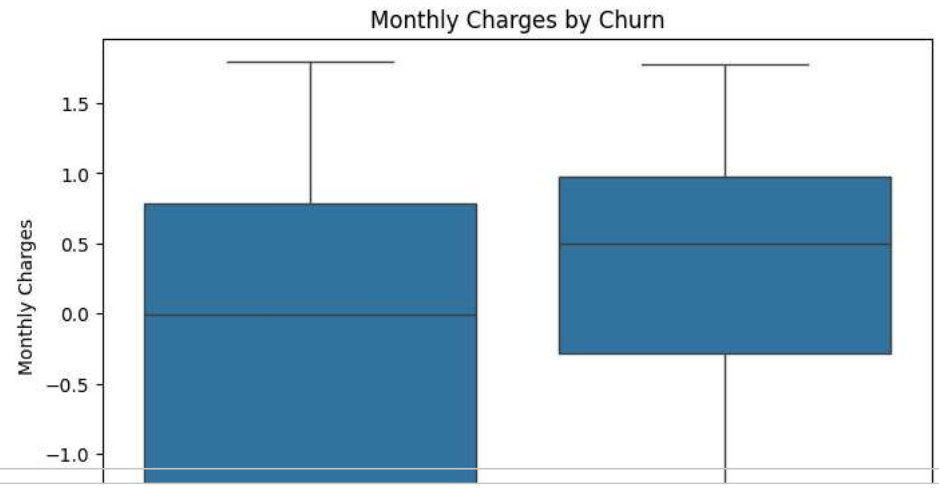

```
#register num:241801041
#name:Darshan SP
plt.figure(figsize=(8, 5))
sns.histplot(df["tenure"], kde=True)
plt.title("Distribution of Tenure")
plt.xlabel("Tenure")
plt.ylabel("Frequency")
plt.show()
```




```
#register num:241801041
#name:Darshan SP
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="Contract", hue="Churn")
plt.title("Contract Type vs Churn")
plt.xlabel("Contract Type")
plt.ylabel("Number of Customers")
plt.show()
```




```
#register num:241801041
#name:Darshan SP
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="Churn", y="MonthlyCharges")
plt.title("Monthly Charges by Churn")
plt.xlabel("Churn")
plt.ylabel("Monthly Charges")
plt.show()
```




```
#register num:241801041
#name:Darshan SP
correlation = df.corr(numeric_only=True)
```

```
#register num:241801041
#name:Darshan SP
plt.figure(figsize=(12, 8))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Correlation Heatmap of Numerical Features")
plt.show()
```



EXP-02(PODS)

```
#register num:241801041
#name:Darshan SP
import pandas as pd

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"

df = pd.read_csv(url)

print("Titanic dataset loaded successfully!")
print("Shape:", df.shape)

display(df.head())
```

```
Titanic dataset loaded successfully!
Shape: (891, 12)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

```
#register num:241801041
#name:Darshan SP
print("Shape of Dataset:")
print(df.shape)

print("\nDataset Information:")
df.info()

print("\nStatistical Description:")
display(df.describe())
```

```
Shape of Dataset:
(891, 12)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

Statistical Description:
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
#register num:241801041
#name:Darshan SP
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")

df["Age"] = imputer.fit_transform(df[["Age"]]).ravel()

print("Missing Age values after imputation:")
print(df["Age"].isnull().sum())
```

```
Missing Age values after imputation:
0
```

```
#register num:241801041
#name:Darshan SP
df["Cabin"] = df["Cabin"].fillna("Unknown")

print("Missing Cabin values:")
print(df["Cabin"].isnull().sum())
```

```
Missing Cabin values:
0
```

```
#register num:241801041
#name:Darshan SP
mode_value = df["Embarked"].mode()[0]

df["Embarked"] = df["Embarked"].fillna(mode_value)

print("Missing Embarked values:")
print(df["Embarked"].isnull().sum())
```

```
Missing Embarked values:
0
```

```
#register num:241801041
```

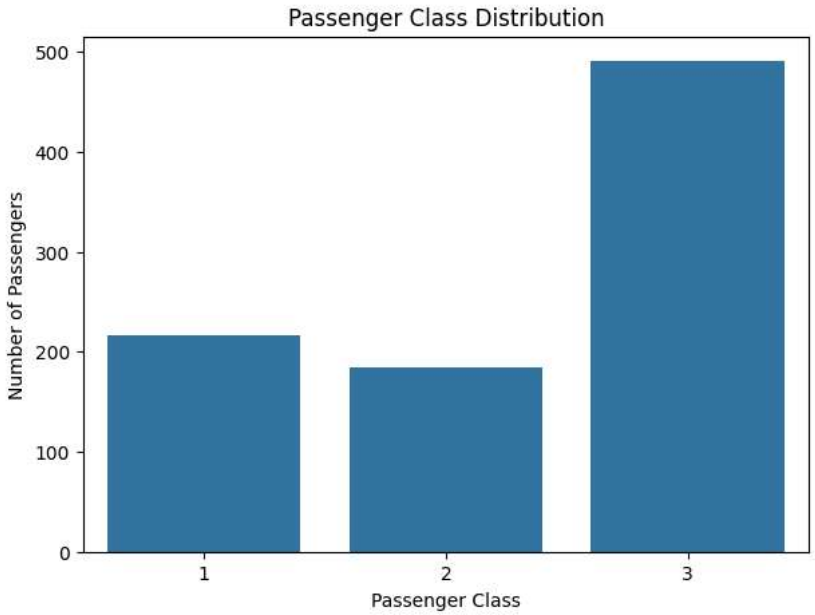
```
#name:Darshan SP
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 5))

sns.countplot(data=df, x="Pclass")

plt.title("Passenger Class Distribution")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")

plt.show()
```




```
#register num:241801041
#name:Darshan SP
female_survived = df[
    (df["Sex"] == "female") &
    (df["Survived"] == 1)
]

display(female_survived[["Name"]])
```

	Name
1	Cumings, Mrs. John Bradley (Florence Briggs Th...
2	Heikkinen, Miss. Laina
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)
8	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9	Nasser, Mrs. Nicholas (Adele Achem)
...	...
874	Abelson, Mrs. Samuel (Hannah Wizosky)
875	Najib, Miss. Adele Kiamie "Jane"
879	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
880	Shelley, Mrs. William (Imanita Parrish Hall)
887	Graham, Miss. Margaret Edith

233 rows × 1 columns

```
#register num:241801041
#name:Darshan SP
third_class_under_18 = df[
    (df["Pclass"] == 3) &
    (df["Age"] < 18)
```

```
]

display(third_class_under_18)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	Unknown	S
10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	PP 9549	16.7000	G6	S
14	15	0	3	Vestrom, Miss. Hulda Amanda Adolfina	female	14.0	0	0	350406	7.8542	Unknown	S
16	17	0	3	Rice, Master. Eugene	male	2.0	4	1	382652	29.1250	Unknown	Q
22	23	1	3	McGowan, Miss. Anna "Annie"	female	15.0	0	0	330923	8.0292	Unknown	Q
...	...	...	...	...	...	...	...	...	...	...	...	...
844	845	0	3	Culumovic, Mr. Jeso	male	17.0	0	0	315090	8.6625	Unknown	S
850	851	0	3	Andersson, Master. Sigvard Harald Elias	male	4.0	4	2	347082	31.2750	Unknown	S
852	853	0	3	Boulos, Miss. Nourelain	female	9.0	1	1	2678	15.2458	Unknown	C

Next steps: Generate code with `third_class_under_18` New interactive sheet

```
#register num:241801041
#name:Darshan SP
class1_older40 = df[
    (df["Pclass"] == 1) &
    (df["Age"] > 40)
]

display(class1_older40)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103	S
35	36	0	1	Holverson, Mr. Alexander Oskar	male	42.0	1	0	113789	52.0000	Unknown	S
52	53	1	1	Harper, Mrs. Henry Sleeper (Myna Haxtun)	female	49.0	1	0	PC 17572	76.7292	D33	C
54	55	0	1	Ostby, Mr. Engelhart Cornelius	male	65.0	0	1	113509	61.9792	B30	C
...	...	...	...	...	...	...	...	...	...	...	...	...
856	857	1	1	Wick, Mrs. George Dennick (Mary Hitchcock)	female	45.0	1	1	36928	164.8667	Unknown	S
857	858	1	1	Daly, Mr. Peter Denis	male	51.0	0	0	113055	26.5500	E17	S
862	863	1	1	Swift, Mrs. Frederick Joel (Margaret Welles Ba...	female	48.0	0	0	17466	25.9292	D17	S

Next steps: Generate code with `class1_older40` New interactive sheet

```
#register num:241801041
#name:Darshan SP
class1_older40_survived = df[
    (df["Pclass"] == 1) &
    (df["Age"] > 40) &
    (df["Survived"] == 1)
]

display(class1_older40_survived)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103	S
52	53	1	1	Harper, Mrs. Henry Sleeper (Myna Haxtun)	female	49.0	1	0	PC 17572	76.7292	D33	C
187	188	1	1	Romaine, Mr. Charles Hallace ("Mr C Rolmane")	male	45.0	0	0	111428	26.5500	Unknown	S
194	195	1	1	Brown, Mrs. James Joseph (Margaret Tobin)	female	44.0	0	0	PC 17610	27.7208	B4	C
195	196	1	1	Lurette, Miss. Elise	female	58.0	0	0	PC 17569	146.5208	B80	C
268	269	1	1	Graham, Mrs. William Thompson (Edith Junkins)	female	58.0	0	1	PC 17582	153.4625	C125	S
275	276	1	1	Andrews, Miss. Kornelia Theodosia	female	63.0	1	0	13502	77.9583	D7	S
299	300	1	1	Baxter, Mrs. James (Helene DeLaudeniere Chaput)	female	50.0	0	1	PC 17558	247.5208	B58 B60	C
337	338	1	1	Burns, Miss. Elizabeth Margaret	female	41.0	0	0	16966	134.5000	E40	C
366	367	1	1	Warren, Mrs. Frank Manley (Anna Sophia Atkinson)	female	60.0	1	0	110813	75.2500	D37	C
380	381	1	1	Bidois, Miss. Rosalie	female	42.0	0	0	PC 17757	227.5250	Unknown	C
449	450	1	1	Peuchen, Major. Arthur Godfrey	male	52.0	0	0	113786	30.5000	C104	S
453	454	1	1	Goldenberg, Mr. Samuel L	male	49.0	1	0	17453	89.1042	C92	C
460	461	1	1	Anderson, Mr. Harry	male	48.0	0	0	19952	26.5500	E12	S
496	497	1	1	Eustis, Miss. Elizabeth Mussey	female	54.0	1	0	36947	78.2667	D20	C
513	514	1	1	Rothschild, Mrs. Martin (Elizabeth L. Barrett)	female	54.0	1	0	PC 17603	59.4000	Unknown	C
523	524	1	1	Hippach, Mrs. Louis Albert (Ida Sophia Fischer)	female	44.0	0	1	111361	57.9792	B18	C
556	557	1	1	Duff Gordon, Lady. (Lucille Christiana Sutherl...	female	48.0	1	0	11755	39.6000	A16	C
571	572	1	1	Appleton, Mrs. Edward Dale (Charlotte Lamson)	female	53.0	2	0	11769	51.4792	C101	S
587	588	1	1	Frolicher-Stehli, Mr. Maxmillian	male	60.0	1	1	13567	79.2000	B41	C
591	592	1	1	Stephenson, Mrs. Walter Bertram (Martha Eustis)	female	52.0	1	0	36947	78.2667	D20	C
599	600	1	1	Duff Gordon, Sir. Cosmo Edmund ("Mr Morgan")	male	49.0	1	0	PC 17485	56.9292	A20	C
621	622	1	1	Kimball, Mr. Edwin Nelson Jr	male	42.0	1	0	11753	52.5542	D19	S
630	631	1	1	Barkworth, Mr. Algernon Henry Wilson	male	80.0	0	0	27042	30.0000	A23	S
645	646	1	1	Harper, Mr. Henry Sleeper	male	48.0	1	0	PC 17572	76.7292	D33	C

Next steps: Generate code with `class1_older40_survived` New interactive sheet

```
#register num:241801041
#name:Darshan SP
male_fare_100 = df[
    (df["Sex"] == "male") &
    (df["Fare"] > 100)
]
```

```
display(male_fare_100)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
27	28	0	1	Fortune, Mr. Charles Alexander	male	19.00	3	2	19950	263.0000	C23 C25 C27	S
118	119	0	1	Baxter, Mr. Quigg Edmond	male	24.00	0	1	PC 17558	247.5208	B58 B60	C
305	306	1	1	Allison, Master. Hudson Trevor	male	0.92	1	2	113781	151.5500	C22 C26	S
332	333	0	1	Graham, Mr. George Edward	male	38.00	0	1	PC 17582	153.4625	C91	S
373	374	0	1	Ringhini, Mr. Sante	male	22.00	0	0	PC 17760	135.6333	Unknown	C
377	378	0	1	Widener, Mr. Harry Elkins	male	27.00	0	2	113503	211.5000	C82	C
390	391	1	1	Carter, Mr. William Ernest	male	36.00	1	2	113760	120.0000	B96 B98	S
438	439	0	1	Fortune, Mr. Mark	male	64.00	1	4	19950	263.0000	C23 C25 C27	S
505	506	0	1	Penasco y Castellana, Mr. Victor de Satode	male	18.00	1	0	PC 17758	108.9000	C65	C
527	528	0	1	Farthing, Mr. John	male	28.00	0	0	PC 17483	221.7792	C95	S
544	545	0	1	Douglas, Mr. Walter Donald	male	50.00	1	0	PC 17761	106.4250	C86	C
550	551	1	1	Thayer, Mr. John Borland Jr	male	17.00	0	2	17421	110.8833	C70	C

Next steps: Generate code with `male_fare_100` New interactive sheet

```
#register num:241801041
#name:Darshan SP
port_c_class2 = df[
    (df["Embarked"] == "C") &
    (df["Pclass"] == 2)
]

display(port_c_class2)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	Unknown	C
43	44	1	2	Laroche, Miss. Simonne Marie Anne Andree	female	3.0	1	2	SC/Paris 2123	41.5792	Unknown	C
122	123	0	2	Nasser, Mr. Nicholas	male	32.5	1	0	237736	30.0708	Unknown	C
135	136	0	2	Richard, Mr. Emile	male	23.0	0	0	SC/PARIS 2133	15.0458	Unknown	C
181	182	0	2	Pernot, Mr. Rene	male	28.0	0	0	SC/PARIS 2131	15.0500	Unknown	C
292	293	0	2	Levy, Mr. Rene Jacques	male	36.0	0	0	SC/Paris 2163	12.8750	D	C
308	309	0	2	Abelson, Mr. Samuel	male	30.0	1	0	P/PP 3381	24.0000	Unknown	C
361	362	0	2	del Carlo, Mr. Sebastiano	male	29.0	1	0	SC/PARIS 2167	27.7208	Unknown	C
389	390	1	2	Lehmann, Miss. Bertha	female	17.0	0	0	SC 1748	12.0000	Unknown	C
473	474	1	2	Jerwan, Mrs. Amin S (Marie Marthe Thuillard)	female	23.0	0	0	SC/AH Basle 541	13.7917	D	C
547	548	1	2	Padro y Manent, Mr. Julian	male	28.0	0	0	SC/PARIS 2146	13.8625	Unknown	C

Next steps: Generate code with port_c_class2 New interactive sheet

```
#register num:241801041
#name:Darshan SP
more_than_2_sibsp = df[df["SibSp"] > 2]

display(more_than_2_sibsp)
```

```
#register num:241801041
#name:Darshan SP
no_survival_no_family = df[
    (df["Survived"] == 0) &
    (df["SibSp"] == 0) &
    (df["Parch"] == 0)
]

display(no_survival_no_family)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
50	51	0	3	Panula, Master. Juha	male	7.0	4	1	3101295	39.6875	Unknown	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	Unknown	S
5	6	0	3	Moran, Mr. James	male	28.0	0	0	330877	8.4583	Unknown	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
12	13	0	3	Saundercock, Mr. William Henry	male	20.0	0	0	A/5. 2151	8.0500	Unknown	S
71	72	0	3	Goodwin, Miss. Lillian Amy	female	16.0	5	2	CA 2144	46.9000	Unknown	S
14	15	0	3	Vestrom, Miss. Hulda Amanda Adolfina	female	14.0	0	0	350406	7.8542	Unknown	S
...	...	...	...	...	...	...	...	...	...	...	...	...
882	883	0	3	Dahlberg, Miss. Gerda Ulrika	female	22.0	0	0	7552	10.5167	Unknown	S
119	120	0	3	Andersson, Miss. Ellis Anna Maria	female	2.0	4	2	347082	31.2750	Unknown	S
883	884	0	2	Banfield, Mr. Frederick James	male	28.0	0	0	C.A. 34068	10.5000	Unknown	S
159	160	0	3	Sage, Master. Thomas Henry	male	28.0	8	2	CA. 2343	69.5500	Unknown	S
164	165	0	3	Panula, Master. Eino Viljami	male	1.0	4	1	3101295	39.6875	Unknown	S

Next steps: Generate code with no_survival_no_family | New interactive sheet

Ex3a: Model Planning and Building(pods)

```
#DARSHAN SP
#241801041
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
```

```
#DARSHAN SP
#241801041
df = pd.read_csv("advertising.csv")

print(df.head())
```

```
      TV  Radio  Newspaper  Sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3   12.0
3  151.5   41.3       58.5   16.5
4  180.8   10.8       58.4   17.9
```

```
#DARSHAN SP
#241801041
print(df.info())
print(df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
None
               TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   15.130500
std     85.854236   14.846809   21.778621    5.283892
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   11.000000
50%    149.750000   22.900000   25.750000   16.000000
75%    218.825000   36.525000   45.100000   19.050000
max    296.400000   49.600000  114.000000   27.000000
```

```
#DARSHAN SP
#241801041
print(df.isnull().sum())
```

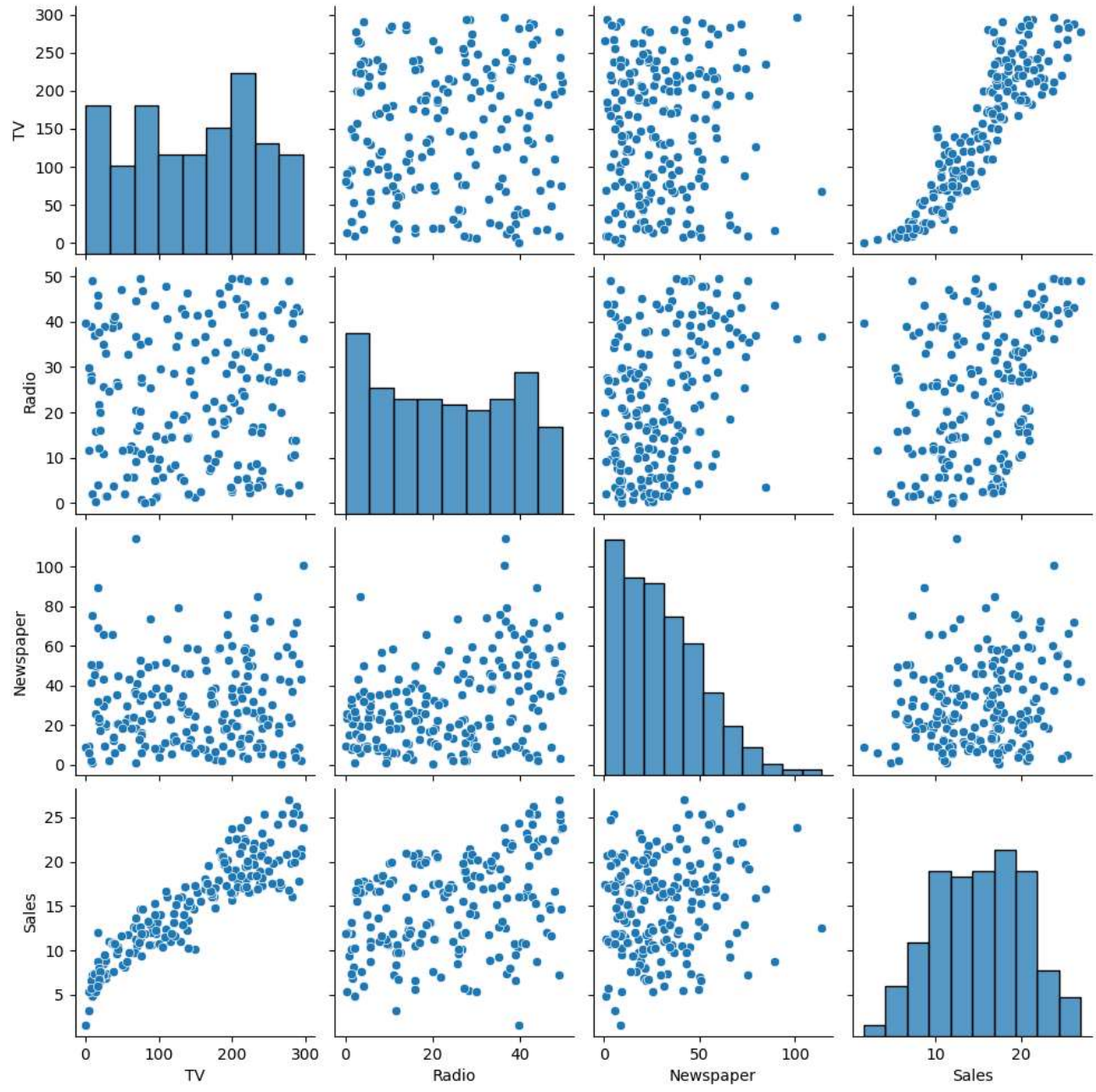
```
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64
```

```
#DARSHAN SP
#241801041
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

print(X.head())
print(y.head())
```

```
      TV  Radio  Newspaper
0  230.1   37.8       69.2
1   44.5   39.3       45.1
2   17.2   45.9       69.3
3  151.5   41.3       58.5
4  180.8   10.8       58.4
0    22.1
1    10.4
2    12.0
3    16.5
4    17.9
Name: Sales, dtype: float64
```

```
#DARSHAN SP
#241801041
sns.pairplot(df)
plt.show()
```




```
#DARSHAN SP
#241801041
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (160, 3)
Testing data: (40, 3)
```

```
#DARSHAN SP
#241801041
lr = LinearRegression()

lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

print(y_pred_lr)
```

```
[17.0347724  20.40974033 23.72398873  9.27278518 21.68271879 12.56940161
 21.08119452  8.69035045 17.23701254 16.66657475  8.92396497  8.4817344
 18.2075123   8.06750728 12.64550975 14.93162809  8.12814594 17.89876565
 11.00880637 20.47832788 20.80631846 12.59883297 10.9051829  22.38854775
  9.41796094  7.92506736 20.83908497 13.81520938 10.77080925  7.92682509
 15.95947357 10.63490851 20.80292008 10.43434164 21.5784752  21.18364487
 12.12821771 22.80953262 12.60992766  6.46441252]
```

```
#DARSHAN SP
#241801041
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)

poly_model.fit(X_train, y_train)

y_pred_poly = poly_model.predict(X_test)

print(y_pred_poly)
```

```
[17.562438   21.96285664 22.07580481  7.34916293 23.23684435 12.37109504
 22.12872506  8.76904874 17.01492635 16.91480608  7.86402513  9.1023732
 17.43691412  5.99987179 13.70960498 15.38247085  6.28283192 18.02070399
 10.95120455 20.20193781 20.93683157 13.01896785  9.65505427 22.48602888
 10.0006458   8.88804873 21.83258724 14.37821146 11.24748724  6.08610266
 16.1977029  10.4879825  22.56616524  9.20046174 19.54142042 21.31849501
 11.27647626 21.73705841 13.18303888  7.12553149]
```

```
#DARSHAN SP
#241801041
svr = SVR(kernel='rbf')

svr.fit(X_train, y_train)

y_pred_svr = svr.predict(X_test)

print(y_pred_svr)
```

```
[17.15330196 19.49875585 20.96245814  8.60209215 19.88553201 12.03803929
 19.98237498  9.22752786 17.73379117 17.08961377  8.99691527  9.3292374
 18.62744348  8.14145921 14.08177689 16.00015525  8.04682867 17.97920438
 11.12951222 19.55761255 20.0541767  12.95980908  9.9170165  20.86239638
 10.43376345  9.07422216 19.89744797 14.71765498 11.47650759  8.24606316
 16.91068571 10.75656165 19.72411632 10.2142965  20.15216829 19.79413534
 10.88647539 20.9688165  13.5160367   7.85022468]
```

```
#DARSHAN SP
#241801041
mae = mean_absolute_error(y_test, y_pred_lr)
mse = mean_squared_error(y_test, y_pred_lr)
rmse = np.sqrt(mse)

print("Linear Regression")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
```

```
Linear Regression
MAE : 1.2748262109549338
MSE : 2.9077569102710896
RMSE: 1.7052146229349223
```

```
#DARSHAN SP
#241801041
mae = mean_absolute_error(y_test, y_pred_poly)
mse = mean_squared_error(y_test, y_pred_poly)
rmse = np.sqrt(mse)

print("Polynomial Regression")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
```

```
Polynomial Regression
MAE : 0.9033713382792442
MSE : 1.44254263690159
RMSE: 1.2010589647896517
```

```
#DARSHAN SP
#241801041
mae = mean_absolute_error(y_test, y_pred_svr)
mse = mean_squared_error(y_test, y_pred_svr)
rmse = np.sqrt(mse)

print("SVR")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
```

```
SVR
MAE : 1.4836223881587627
MSE : 3.499087197255254
RMSE: 1.8705847206836834
```

```
#DARSHAN SP
#241801041
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

kmeans.fit(X)

df['Cluster'] = kmeans.labels_

print(df.head())
```

```
      TV  Radio  Newspaper  Sales  Cluster
0  230.1   37.8       69.2   22.1        2
1   44.5   39.3       45.1   10.4        1
2   17.2   45.9       69.3   12.0        1
3  151.5   41.3       58.5   16.5        0
4  180.8   10.8       58.4   17.9        0
```

```
#DARSHAN SP
#241801041
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
```

```
kmeans.fit(X)

df['Cluster'] = kmeans.labels_

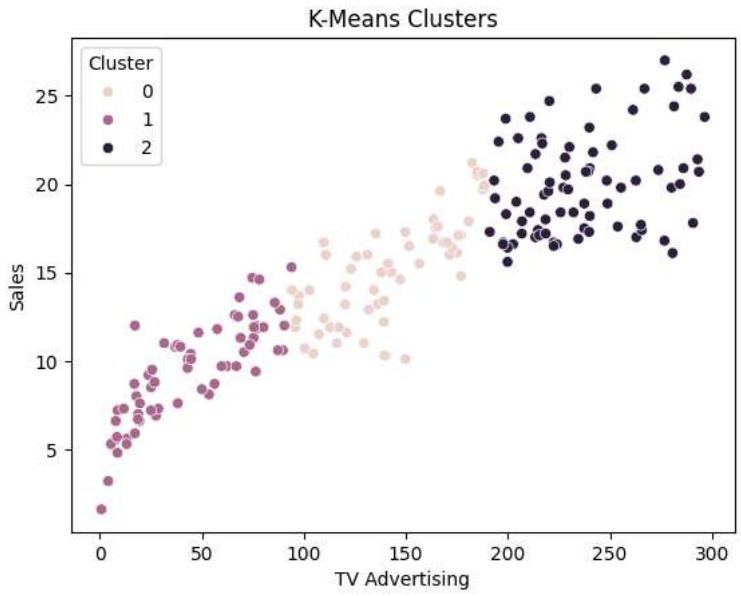
print(df.head())
```

```
      TV  Radio  Newspaper  Sales  Cluster
0  230.1   37.8       69.2   22.1        2
1   44.5   39.3       45.1   10.4        1
2   17.2   45.9       69.3   12.0        1
3  151.5   41.3       58.5   16.5        0
4  180.8   10.8       58.4   17.9        0
```

```
#DARSHAN SP
#241801041
sns.scatterplot(
    data=df,
    x='TV',
    y='Sales',
    hue='Cluster'
)

plt.xlabel("TV Advertising")
plt.ylabel("Sales")
plt.title("K-Means Clusters")

plt.show()
```




```
#DARSHAN SP
#241801041
print(kmeans.cluster_centers_)
```

```
[[141.79655172  20.77931034  26.18103448]
 [ 45.20923077  23.42769231  31.06461538]
 [236.95714286  24.9974026   33.41688312]]
```

exp 3 b (PODs)

```
#DARSHAN SP
#241801041
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
```

```
#DARSHAN SP
#241801041
import requests
url = "https://raw.githubusercontent.com/ageron/handson-ml/master/datasets/housing/housing.csv"
response = requests.get(url)
response.raise_for_status()
with open("house_prices.csv", "wb") as f:
    f.write(response.content)
print("house_prices.csv downloaded successfully!")
```

```
house_prices.csv downloaded successfully!
```

```
#DARSHAN SP
#241801041
df = pd.read_csv("house_prices.csv")
print("First 5 rows:")
print(df.head())
```

```
First 5 rows:
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23     37.88                41.0        880.0           129.0
1    -122.22     37.86                21.0       7099.0          1106.0
2    -122.24     37.85                52.0       1467.0           190.0
3    -122.25     37.85                52.0       1274.0           235.0
4    -122.25     37.85                52.0       1627.0           280.0

   population  households  median_income  median_house_value ocean_proximity
0       322.0       126.0         8.3252            452600.0        NEAR BAY
1      2401.0      1138.0         8.3014            358500.0        NEAR BAY
2       496.0       177.0         7.2574            352100.0        NEAR BAY
3       558.0       219.0         5.6431            341300.0        NEAR BAY
4       565.0       259.0         3.8462            342200.0        NEAR BAY
```

```
#DARSHAN SP
#241801041
print("\nDataset Information:")
print(df.info())
print("\nStatistical Description:")
print(df.describe())
```

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

```
None

Statistical Description:
          longitude      latitude  housing_median_age   total_rooms  \
count  20640.000000  20640.000000        20640.000000  20640.000000
mean    -119.569704     35.631861           28.639486   2635.763081
std        2.003532      2.135952           12.585558   2181.615252
min     -124.350000     32.540000            1.000000      2.000000
25%     -121.800000     33.930000           18.000000   1447.750000
50%     -118.490000     34.260000           29.000000   2127.000000
75%     -118.010000     37.710000           37.000000   3148.000000
max     -114.310000     41.950000           52.000000  39320.000000

       total_bedrooms    population    households  median_income  \
count    20433.000000  20640.000000  20640.000000   20640.000000
mean       537.870553   1425.476744    499.539680       3.870671
std        421.385070   1132.462122    382.329753       1.899822
min          1.000000      3.000000      1.000000       0.499900
25%        296.000000    787.000000    280.000000       2.563400
50%        435.000000   1166.000000    409.000000       3.534800
75%        647.000000   1725.000000    605.000000       4.743250
max       6445.000000  35682.000000   6082.000000      15.000100

       median_house_value
count        20640.000000
mean        206855.816909
std         115395.615874
min          14999.000000
25%         119600.000000
50%         179700.000000
75%         264725.000000
max         500001.000000
```

```
#DARSHAN SP
#241801041
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
longitude               0
latitude                0
housing_median_age      0
total_rooms             0
total_bedrooms        207
population              0
households              0
median_income           0
median_house_value      0
ocean_proximity         0
dtype: int64
```

```
#DARSHAN SP
#241801041
df['total_bedrooms'].fillna(df['total_bedrooms'].median(), inplace=True)
print("Missing values after handling:")
print(df.isnull().sum())
```

```
Missing values after handling:
longitude             0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        0
population            0
households            0
median_income         0
median_house_value    0
ocean_proximity       0
dtype: int64
/tmp/ipykernel_5332/397271792.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chaine
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df['total_bedrooms'].fillna(df['total_bedrooms'].median(), inplace=True)
```

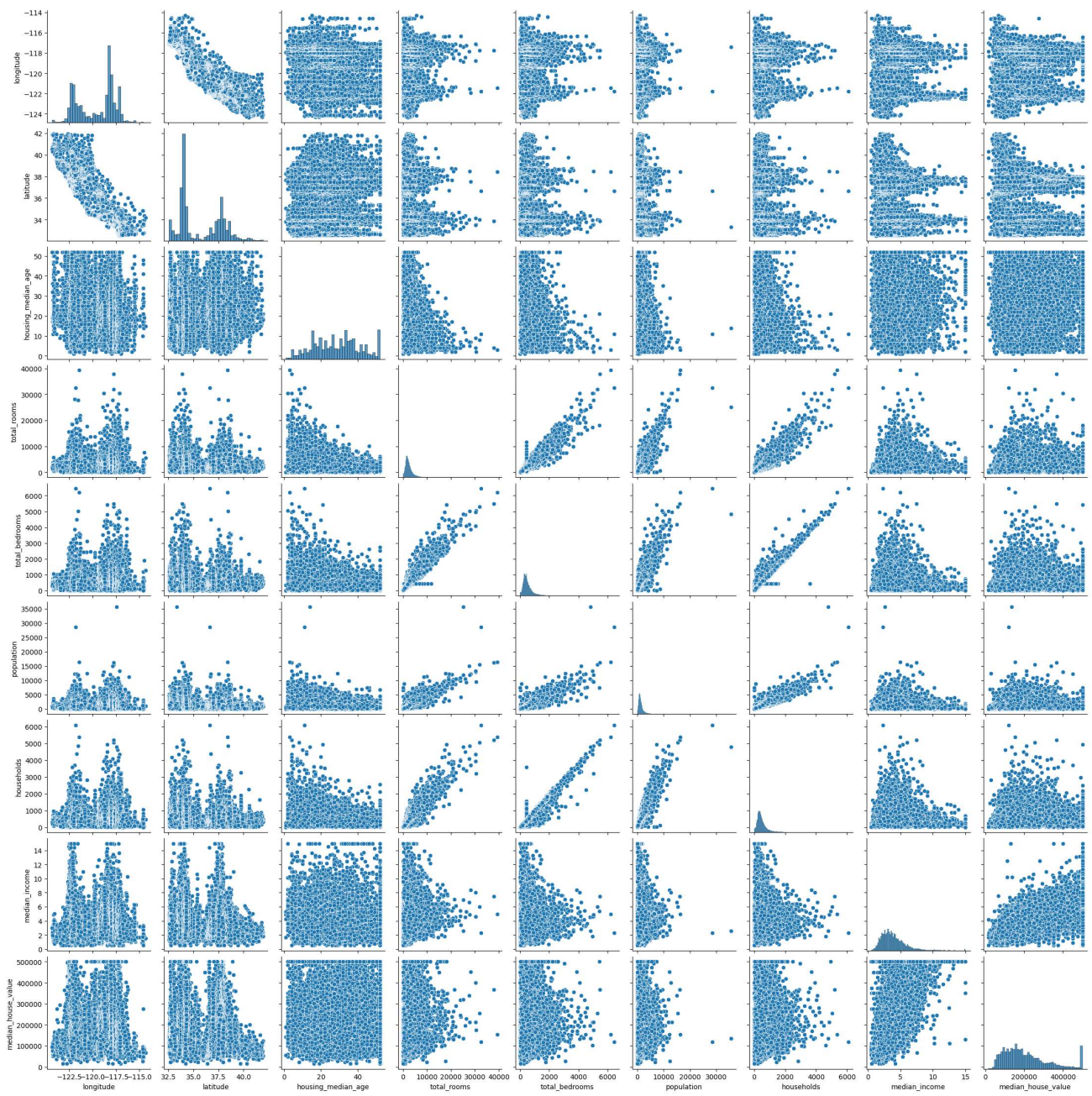
```
#DARSHAN SP
#241801041
```

```
X = df[['housing_median_age', 'total_rooms', 'total_bedrooms', 'median_income']]
y = df['median_house_value']
print("\nFeatures:")
print(X.head())
print("\nTarget:")
print(y.head())
```

```
Features:
   housing_median_age  total_rooms  total_bedrooms  median_income
0                41.0        880.0           129.0         8.3252
1                21.0       7099.0          1106.0         8.3014
2                52.0       1467.0           190.0         7.2574
3                52.0       1274.0           235.0         5.6431
4                52.0       1627.0           280.0         3.8462

Target:
0    452600.0
1    358500.0
2    352100.0
3    341300.0
4    342200.0
Name: median_house_value, dtype: float64
```

```
#DARSHAN SP
#241801041
sns.pairplot(df)
plt.show()
```




```
#DARSHAN SP
#241801041
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    y,
    test_size=0.2,
    random_state=42
)
```

```
#DARSHAN SP
#241801041
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)
```

```
#DARSHAN SP
#241801041
polynomial_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
polynomial_model.fit(X_train, y_train)
y_pred_poly = polynomial_model.predict(X_test)
```

```
#DARSHAN SP
#241801041
svr_model = SVR(kernel='rbf')
svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)
```

```
#DARSHAN SP
#241801041
models = {
    "Linear Regression": y_pred_linear,
    "Polynomial Regression": y_pred_poly,
    "SVR": y_pred_svr
}
for name, prediction in models.items():
    mae = mean_absolute_error(y_test, prediction)
    mse = mean_squared_error(y_test, prediction)
    rmse = np.sqrt(mse)
    print("\nModel:", name)
    print("MAE :", mae)
    print("MSE :", mse)
    print("RMSE:", rmse)
```

```
Model: Linear Regression
MAE : 58342.91635674606
MSE : 6356567901.161352
RMSE: 79728.0872789593

Model: Polynomial Regression
MAE : 57470.90597882158
MSE : 6791633037.7910385
RMSE: 82411.36473685555

Model: SVR
MAE : 87333.38074091132
MSE : 13739650709.806314
RMSE: 117216.25616699381
```

```
#DARSHAN SP
#241801041
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
kmeans.fit(X)
df['Cluster'] = kmeans.labels_
print("\nCluster Labels:")
print(df[['housing_median_age', 'total_rooms', 'total_bedrooms', 'median_income', 'Cluster']])
```

```
Cluster Labels:
       housing_median_age  total_rooms  total_bedrooms  median_income  Cluster
0                    41.0        880.0           129.0         8.3252        1
```

```
1                   21.0       7099.0          1106.0          8.3014        0
2                   52.0       1467.0           190.0          7.2574        1
3                   52.0       1274.0           235.0          5.6431        1
4                   52.0       1627.0           280.0          3.8462        1
...                  ...          ...             ...             ...      ...
20635               25.0       1665.0           374.0          1.5603        1
20636               18.0        697.0           150.0          2.5568        1
20637               17.0       2254.0           485.0          1.7000        1
20638               18.0       1860.0           409.0          1.8672        1
20639               16.0       2785.0           616.0          2.3886        1

[20640 rows x 5 columns]
```

```
#DARSHAN SP
#241801041
sns.scatterplot(
    data=df,
    x='total_rooms',
    y='median_house_value',
    hue='Cluster'
)
```

```
#241801041
#DARSHAN SP
# CELL 1: Import libraries and load dataset

import subprocess
import sys
import importlib.util

if importlib.util.find_spec("nltk") is None:
    subprocess.check_call([
        sys.executable, "-m", "pip", "install", "-q", "nltk"
    ])

import pandas as pd
import numpy as np
import nltk
import re
import string

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
nltk.download("stopwords", quiet=True)

# Download dataset automatically
url = (
    "https://huggingface.co/datasets/"
    "PJ2005/amazon-fine-food-reviews/resolve/main/Reviews.csv"
)

df = pd.read_csv(url, nrows=10000)

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset loaded successfully!
Dataset shape: (10000, 10)
```

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	Helpf
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0	
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	1	
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	3	
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	0	

```
#241801041
#DARSHAN SP
# CELL 2: Dataset cleaning

df = df[["Text"]].dropna().copy()
df["Text"] = df["Text"].astype(str).str.strip()
df = df[df["Text"] != ""]
df = df.drop_duplicates(subset=["Text"])
df = df.head(10000).reset_index(drop=True)

print("Cleaned dataset shape:", df.shape)
print("\nFirst five reviews:")
```

```
display(df.head())
```

```
Cleaned dataset shape: (9513, 1)

First five reviews:
```

	Text
0	I have bought several of the Vitality canned d...
1	Product arrived labeled as Jumbo Salted Peanut...
2	This is a confection that has been around a fe...
3	If you are looking for the secret ingredient i...
4	Great taffy at a great price. There was a wid...

```
#241801041
#DARSHAN SP
# CELL 3: Text preprocessing

stop_words = set(stopwords.words("english"))

def preprocess_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"[^a-z\s]", " ", text)

    tokens = word_tokenize(text)

    tokens = [
        word for word in tokens
        if word not in stop_words and len(word) > 1
    ]

    return " ".join(tokens)

# Apply preprocessing
df["Cleaned_Text"] = df["Text"].apply(preprocess_text)

print("Original and cleaned reviews:")
display(df[["Text", "Cleaned_Text"]].head(5))
```

```
Original and cleaned reviews:
```

	Text	Cleaned_Text
0	I have bought several of the Vitality canned d...	bought several vitality canned dog food produc...
1	Product arrived labeled as Jumbo Salted Peanut...	product arrived labeled jumbo salted peanuts p...
2	This is a confection that has been around a fe...	confection around centuries light pillowy citr...
3	If you are looking for the secret ingredient i...	looking secret ingredient robitussin believe f...
4	Great taffy at a great price. There was a wid...	great taffy great price wide assortment yummy ...

```
#241801041
#DARSHAN SP
# CELL 4: TF-IDF vectorization

vectorizer = TfidfVectorizer(max_features=5000)

tfidf_matrix = vectorizer.fit_transform(df["Cleaned_Text"])

print("TF-IDF matrix shape:", tfidf_matrix.shape)
print("Number of documents:", tfidf_matrix.shape[0])
print("Vocabulary size:", len(vectorizer.get_feature_names_out()))

print("\nSample vocabulary:")
print(vectorizer.get_feature_names_out()[:20])
```

```
TF-IDF matrix shape: (9513, 5000)
Number of documents: 9513
Vocabulary size: 5000

Sample vocabulary:
['ability' 'able' 'absolute' 'absolutely' 'absorb' 'absorbed' 'acacia'
 'acai' 'accept' 'acceptable' 'access' 'accident' 'accidentally'
 'according' 'account' 'accurate' 'accustomed' 'acerola' 'ache' 'aches']
```

```
#241801041
#DARSHAN SP
# CELL 5: Document retrieval function

def retrieve_reviews(query, top_n=5):
    cleaned_query = preprocess_text(query)

    if not cleaned_query.strip():
        print("Please enter a meaningful search query.")
```

```
        return pd.DataFrame()

    query_vector = vectorizer.transform([cleaned_query])

    similarities = cosine_similarity(
        query_vector, tfidf_matrix
    ).flatten()

    top_indices = np.argsort(similarities)[::-1][:top_n]

    results = df.iloc[top_indices][["Text", "Cleaned_Text"]].copy()
    results["Similarity_Score"] = similarities[top_indices]

    results = results[results["Similarity_Score"] > 0]

    return results.reset_index(drop=True)

print("Information retrieval system is ready!")
```

```
Information retrieval system is ready!
```

```
#241801041
#DARSHAN SP
# CELL 6: Test with sample queries

queries = [
    "great product with fast shipping",
    "disappointed",
    "excellent taste",
    "poor packaging"
]

for query in queries:
    print("\n" + "=" * 70)
    print("Search Query:", query)
    print("=" * 70)

    results = retrieve_reviews(query, top_n=5)

    if results.empty:
        print("No matching reviews found.")
    else:
        for i, row in results.iterrows():
            print(f"\nResult {i + 1}")
            print("Similarity Score:", round(row["Similarity_Score"], 4
            print("Review:", row["Text"][:500])
```

```
 Score: 0.3419
is coffee is good and has nice subtle flavor. I am disappointed though because


 Score: 0.3211
was disappointed in this product because I thought it would be bigger.<br />Als

============================================================
ry: excellent taste
============================================================


 Score: 0.498
 is very very good sweets<br />Amazon always is excellent<br />Amazon always is


 Score: 0.4607
e Popchips were excellent - had excellent flavor and low in calories.  When you


 Score: 0.406
cellent product. Exactly what they said it would be. Superior chamomile from wh


 Score: 0.3859
 recieved the product on time and in excellent shape. It tastes good and meets


 Score: 0.3829
tle says it all, a Excellent quality product and Excellent service at a fair pr

============================================================
ry: poor packaging
============================================================


 Score: 0.3611
dnt really enjoy these after wanting to try something other than aunt jemima's


 Score: 0.3332
e soup is rich and tasty, but DON'T buy it from Amazon.  The packaging is so po


 Score: 0.3265
was delighted to find this product online, because I cannot always find it in t


 Score: 0.3091
is review is for the packaging and not the actual mix, since the mix is a very


 Score: 0.2707
would agree with the other poor ratings for Royal Oak - very poor customer serv
```

```
#241801041
#DARSHAN SP
# CELL 7: Compare query results

comparison_queries = [
    "great product with fast shipping",
    "disappointed",
    "excellent taste",
    "poor packaging"
]

comparison_results = []

for query in comparison_queries:
    results = retrieve_reviews(query, top_n=5)

    for _, row in results.iterrows():
        comparison_results.append({
            "Query": query,
            "Review": row["Text"],
            "Similarity_Score": round(
                row["Similarity_Score"], 4
            )
        })

comparison_df = pd.DataFrame(comparison_results)

print("Query evaluation results:")
display(comparison_df)
```

```
#241801041
#DARSHAN SP
# CELL 8: Save output files

df.to_csv("cleaned_reviews.csv", index=False)
comparison_df.to_csv("retrieval_results.csv", index=False)

print("Files saved successfully:")
print("1. cleaned_reviews.csv")
print("2. retrieval_results.csv")

from google.colab import files

files.download("cleaned_reviews.csv")
```

```
#241801041
#DARSHAN SP
import pandas as pd

url = (
    "https://huggingface.co/datasets/"
    "PJ2005/amazon-fine-food-reviews/resolve/main/Reviews.csv"
)

df = pd.read_csv(url, nrows=10000)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
print(df.head())
print("Columns:", df.columns.tolist())
```

```
Dataset loaded successfully!
Shape: (10000, 10)
   Id   ProductId          UserId                      ProfileName  \
0   1  B001E4KFG0  A3SGXH7AUHU8GW                       delmartian
1   2  B00813GRG4  A1D87F6ZCVE5NK                           dll pa
2   3  B000LQOCH0   ABXLMWJIXXAIN  Natalia Corres "Natalia Corres"
3   4  B000UA0QIQ  A395BORC6FGVXV                             Karl
4   5  B006K2ZZ7K  A1UQRSCLF8GW1T    Michael D. Bigham "M. Wassir"

   HelpfulnessNumerator  HelpfulnessDenominator  Score        Time  \
0                     1                       1      5  1303862400
1                     0                       0      1  1346976000
2                     1                       1      4  1219017600
3                     3                       3      2  1307923200
4                     0                       0      5  1350777600

                Summary                                                Text
0  Good Quality Dog Food  I have bought several of the Vitality canned d...
1      Not as Advertised  Product arrived labeled as Jumbo Salted Peanut...
2  "Delight" says it all  This is a confection that has been around a fe...
3         Cough Medicine  If you are looking for the secret ingredient i...
4            Great taffy  Great taffy at a great price.  There was a wid...
Columns: ['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator', 'HelpfulnessDenomina
```

```
#241801041
#DARSHAN SP
import spacy
from collections import Counter
import pandas as pd

try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    !python -m spacy download en_core_web_sm
    nlp = spacy.load("en_core_web_sm")

print("spaCy model loaded successfully!")
```

```
spaCy model loaded successfully!
```

```
#241801041
#DARSHAN SP
```

```
df = df[["Text"]].dropna().copy()
df["Text"] = df["Text"].astype(str).str.strip()
df = df[df["Text"] != ""]
df = df.drop_duplicates(subset=["Text"])
df = df.head(10000).reset_index(drop=True)

print("Cleaned dataset shape:", df.shape)
print(df.head())
```

```
Cleaned dataset shape: (9513, 1)
                                                Text
0  I have bought several of the Vitality canned d...
1  Product arrived labeled as Jumbo Salted Peanut...
2  This is a confection that has been around a fe...
3  If you are looking for the secret ingredient i...
4  Great taffy at a great price.  There was a wid...
```

```
#241801041
#DARSHAN SP
for i, review in enumerate(df["Text"].head(5), start=1):
    doc = nlp(review)

    print(f"\nReview {i}:")
    print("Original Review:", review)
    print("Tokens:", [token.text for token in doc])
    print("Lemmas:", [token.lemma_ for token in doc])
    print("POS Tags:", [(token.text, token.pos_) for token in doc])
```

```
Review 1:
Original Review: I have bought several of the Vitality canned dog food products and have found the
Tokens: ['I', 'have', 'bought', 'several', 'of', 'the', 'Vitality', 'canned', 'dog', 'food', 'prod
Lemmas: ['I', 'have', 'buy', 'several', 'of', 'the', 'Vitality', 'can', 'dog', 'food', 'product',
POS Tags: [('I', 'PRON'), ('have', 'AUX'), ('bought', 'VERB'), ('several', 'ADJ'), ('of', 'ADP'),

Review 2:
Original Review: Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small
Tokens: ['Product', 'arrived', 'labeled', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanu
Lemmas: ['product', 'arrived', 'label', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanut'
POS Tags: [('Product', 'NOUN'), ('arrived', 'AUX'), ('labeled', 'VERB'), ('as', 'ADP'), ('Jumbo',

Review 3:
Original Review: This is a confection that has been around a few centuries.  It is a light, pillow
Tokens: ['This', 'is', 'a', 'confection', 'that', 'has', 'been', 'around', 'a', 'few', 'centuries'
Lemmas: ['this', 'be', 'a', 'confection', 'that', 'have', 'be', 'around', 'a', 'few', 'century', '
POS Tags: [('This', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('confection', 'NOUN'), ('that', 'PRON')

Review 4:
Original Review: If you are looking for the secret ingredient in Robitussin I believe I have found
Tokens: ['If', 'you', 'are', 'looking', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin',
Lemmas: ['if', 'you', 'be', 'look', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I',
POS Tags: [('If', 'SCONJ'), ('you', 'PRON'), ('are', 'AUX'), ('looking', 'VERB'), ('for', 'ADP'),

Review 5:
Original Review: Great taffy at a great price.  There was a wide assortment of yummy taffy.  Deliv
Tokens: ['Great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'There', 'was', 'a', 'wide', 'as
Lemmas: ['great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'there', 'be', 'a', 'wide', 'ass
POS Tags: [('Great', 'ADJ'), ('taffy', 'NOUN'), ('at', 'ADP'), ('a', 'DET'), ('great', 'ADJ'), ('p
```

```
#241801041
#DARSHAN SP
pos_counter = Counter()
```

```
for doc in nlp.pipe(df["Text"].tolist(), batch_size=100):
    for token in doc:
        if not token.is_space and not token.is_punct:
            pos_counter[token.pos_] += 1

print("POS Category Frequencies:")
for tag in ["NOUN", "VERB", "ADJ", "ADV"]:
    print(f"{tag}: {pos_counter[tag]}")

print("\nAll POS categories:")
for tag, count in pos_counter.most_common():
    print(f"{tag}: {count}")
```

```
POS Category Frequencies:
NOUN: 137241
VERB: 91165
ADJ: 68120
ADV: 48256

All POS categories:
NOUN: 137241
PRON: 96008
VERB: 91165
ADJ: 68120
ADP: 66029
DET: 65365
AUX: 54350
ADV: 48256
PROPN: 35168
CCONJ: 30542
PART: 23682
SCONJ: 15802
NUM: 11157
PUNCT: 4410
X: 2647
INTJ: 1644
SYM: 799
```

```
#241801041
#DARSHAN SP
entity_records = []

for review_id, doc in enumerate(
    nlp.pipe(df["Text"].tolist(), batch_size=100), start=1
):
    for ent in doc.ents:
        entity_records.append({
            "Review_ID": review_id,
            "Entity": ent.text,
            "Entity_Label": ent.label_
        })

entities_df = pd.DataFrame(
    entity_records,
    columns=["Review_ID", "Entity", "Entity_Label"]
)

print("First 20 extracted entities:")
print(entities_df.head(20))

print("\nEntities from the first five reviews:")
```

```
for i, review in enumerate(df["Text"].head(5), start=1):
    doc = nlp(review)
    print(f"\nReview {i}:")
    for ent in doc.ents:
        print(ent.text, ":", ent.label_)
```

```
First 20 extracted entities:
    Review_ID                            Entity Entity_Label
0           1                          Vitality          ORG
1           1                        My Labrador       PERSON
2           2                       Jumbo Salted          ORG
3           2                              Jumbo  WORK_OF_ART
4           3          around a few centuries         DATE
5           3          pillowy citrus gelatin          ORG
6           3                        Filberts          GPE
7           3                      C.S. Lewis'          ORG
8           3             The Lion, The Witch  WORK_OF_ART
9           3                          Edmund      PRODUCT
10          3                         Sisters       PERSON
11          3                           Witch          ORG
12          4                       Robitussin          GPE
13          4        the Root Beer Extract I          LAW
14          5                        Delivery       PERSON
15          6                      five pound     QUANTITY
16          6                   only two weeks         DATE
17          7                      Fralinger's          ORG
18          9                       Wheatgrass       PERSON
19         11  Tequila Picante Gourmet de Inclan          ORG

Entities from the first five reviews:

Review 1:
Vitality : ORG
My Labrador : PERSON

Review 2:
Jumbo Salted : ORG
Jumbo : WORK_OF_ART

Review 3:
around a few centuries : DATE
pillowy citrus gelatin : ORG
Filberts : GPE
C.S. Lewis' : ORG
The Lion, The Witch : WORK_OF_ART
Edmund : PRODUCT
Sisters : PERSON
Witch : ORG

Review 4:
Robitussin : GPE
the Root Beer Extract I : LAW

Review 5:
Delivery : PERSON
```

```
#241801041
#DARSHAN SP
def extract_information(review):
    doc = nlp(review)

    return {
        "Original_Review": review,
        "Tokens": [t.text for t in doc if not t.is_space],
        "Lemmas": [t.lemma_ for t in doc if not t.is_space],
```

```
        "POS_Tags": [
            {"token": t.text, "pos": t.pos_, "tag": t.tag_}
            for t in doc if not t.is_space
        ],
        "Named_Entities": [
            {"entity": ent.text, "label": ent.label_}
            for ent in doc.ents
        ]
    }

test_reviews = [
    "I bought organic honey from Amazon for 250 rupees.",
    "The chocolate product from Nestle tastes very good.",
    "I received the coffee package in Chennai yesterday."
]

test_results = []

for i, review in enumerate(test_reviews, start=1):
    result = extract_information(review)
    test_results.append(result)

    print(f"\nTest Review {i}:")
    print("Original Review:", result["Original_Review"])
    print("Tokens:", result["Tokens"])
    print("Lemmas:", result["Lemmas"])
    print("POS Tags:", result["POS_Tags"])
    print("Named Entities:", result["Named_Entities"])
```

```
Test Review 1:
Original Review: I bought organic honey from Amazon for 250 rupees.
Tokens: ['I', 'bought', 'organic', 'honey', 'from', 'Amazon', 'for', '250', 'rupees', '.']
Lemmas: ['I', 'buy', 'organic', 'honey', 'from', 'Amazon', 'for', '250', 'rupee', '.']
POS Tags: [{'token': 'I', 'pos': 'PRON', 'tag': 'PRP'}, {'token': 'bought', 'pos': 'VERB', 'tag':
Named Entities: [{'entity': 'Amazon', 'label': 'ORG'}, {'entity': '250', 'label': 'CARDINAL'}]

Test Review 2:
Original Review: The chocolate product from Nestle tastes very good.
Tokens: ['The', 'chocolate', 'product', 'from', 'Nestle', 'tastes', 'very', 'good', '.']
Lemmas: ['the', 'chocolate', 'product', 'from', 'Nestle', 'taste', 'very', 'good', '.']
POS Tags: [{'token': 'The', 'pos': 'DET', 'tag': 'DT'}, {'token': 'chocolate', 'pos': 'NOUN', 'tag
Named Entities: [{'entity': 'Nestle', 'label': 'GPE'}]

Test Review 3:
Original Review: I received the coffee package in Chennai yesterday.
Tokens: ['I', 'received', 'the', 'coffee', 'package', 'in', 'Chennai', 'yesterday', '.']
Lemmas: ['I', 'receive', 'the', 'coffee', 'package', 'in', 'Chennai', 'yesterday', '.']
POS Tags: [{'token': 'I', 'pos': 'PRON', 'tag': 'PRP'}, {'token': 'received', 'pos': 'VERB', 'tag'
Named Entities: [{'entity': 'Chennai', 'label': 'GPE'}, {'entity': 'yesterday', 'label': 'DATE'}]
```

```
#241801041
#DARSHAN SP
df.to_csv("cleaned_reviews.csv", index=False)
entities_df.to_csv("extracted_entities.csv", index=False)

structured_df = pd.DataFrame(test_results)
structured_df.to_json(
    "structured_information.json",
    orient="records",
    indent=4,
    force_ascii=False
```

```
)

print("Experiment completed successfully!")

from google.colab import files

files.download("cleaned_reviews.csv")
files.download("extracted_entities.csv")
files.download("structured_information.json")
```

```
Experiment completed successfully!
```

```
#241801041
#Darshan sp
# EX 6a: EXPLORATORY DATA ANALYSIS ON NETFLIX TITLES
# STEP 1: Import libraries
import subprocess
import sys
import importlib.util
required = ["pandas", "numpy", "matplotlib", "seaborn", "wordcloud"]
missing = [p for p in required if importlib.util.find_spec(p) is None]
if missing:
    subprocess.check_call([
        sys.executable, "-m", "pip", "install", "-q", *missing
    ])
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from itertools import combinations
from collections import Counter
from IPython.display import display
import warnings
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")

#241801041
#Darshan sp
# STEP 2: Download dataset and convert it to DataFrame
url = (
    "https://raw.githubusercontent.com/"
    "rfordatascience/tidytuesday/main/data/"
    "2021/2021-04-20/netflix_titles.csv"
)
df = pd.read_csv(url)
print("Dataset downloaded successfully!")
print("Dataset shape:", df.shape)
print("\nFirst five records:")
display(df.head())
print("\nDataset information:")
df.info()
print("\nMissing values:")
print(df.isnull().sum())

#241801041
#Darshan sp
# STEP 3: Clean the dataset
df = df.drop_duplicates(subset=["show_id"]).copy()
text_columns = [
    "type", "title", "director", "cast", "country",
    "rating", "duration", "listed_in", "description"
]
for col in text_columns:
    if col in df.columns:
        df[col] = df[col].fillna("Unknown").astype(str).str.strip()
df["release_year"] = pd.to_numeric(
    df["release_year"], errors="coerce"
)
df["date_added"] = pd.to_datetime(
    df["date_added"], errors="coerce"
)
df["year_added"] = df["date_added"].dt.year
df["month_added"] = df["date_added"].dt.month_name()
print("\nCleaned dataset shape:", df.shape)
display(df.head())

#241801041
#Darshan sp
# STEP 4: Content distribution analysis
print("\n========== CONTENT DISTRIBUTION ==========")
print("\nMovies and TV Shows:")
print(df["type"].value_counts())
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="type")
plt.title("Movies vs TV Shows")
plt.xlabel("Content Type")
```

```
plt.ylabel("Number of Titles")
plt.tight_layout()
plt.show()
print("\nRating distribution:")
print(df["rating"].value_counts())
plt.figure(figsize=(10, 5))
df["rating"].value_counts().plot(kind="bar")
plt.title("Distribution of Netflix Ratings")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
countries = (
    df[df["country"] != "Unknown"]
    .assign(country=lambda x: x["country"].str.split(", "))
    .explode("country")
)
top_countries = countries["country"].value_counts().head(10)
print("\nTop 10 countries:")
print(top_countries)
plt.figure(figsize=(10, 6))
top_countries.sort_values().plot(kind="barh")
plt.title("Top 10 Content-Producing Countries")
plt.xlabel("Number of Titles")
plt.tight_layout()
plt.show()

#241801041
#Darshan sp
# STEP 5: Time-based trend analysis
print("\n========== TIME-BASED TRENDS ==========")
release_trend = df["release_year"].dropna().astype(int).value_counts()
release_trend = release_trend.sort_index()
plt.figure(figsize=(12, 5))
release_trend.plot()
plt.title("Netflix Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.tight_layout()
plt.show()
added_trend = df["year_added"].dropna().astype(int).value_counts()
added_trend = added_trend.sort_index()
plt.figure(figsize=(12, 5))
added_trend.plot(marker="o")
plt.title("Netflix Content Addition Trend")
plt.xlabel("Year Added")
plt.ylabel("Number of Titles")
plt.tight_layout()
plt.show()
print("\nTitles released per year (latest 15 years):")
print(release_trend.tail(15))
print("\nTitles added per year:")
print(added_trend)

#241801041
#Darshan sp
# STEP 6: Duration and content type analysis
print("\n========== DURATION ANALYSIS ==========")
df["duration_value"] = pd.to_numeric(
    df["duration"].str.extract(r"(\d+)")[0],
    errors="coerce"
)
movies = df[df["type"] == "Movie"].copy()
tv_shows = df[df["type"] == "TV Show"].copy()
print("\nMovie duration statistics in minutes:")
print(movies["duration_value"].describe())
print("\nTV show season statistics:")
print(tv_shows["duration_value"].describe())
plt.figure(figsize=(10, 5))
sns.histplot(
    movies["duration_value"].dropna(),
    bins=30, kde=True
)
plt.title("Distribution of Movie Durations")
plt.xlabel("Duration (minutes)")
plt.ylabel("Number of Movies")
plt.tight_layout()
```

```
plt.show()
plt.figure(figsize=(8, 5))
tv_shows["duration_value"].value_counts().sort_index().plot(
    kind="bar"
)
plt.title("Number of Seasons in TV Shows")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.tight_layout()
plt.show()

#241801041
#Darshan sp
# STEP 7: Genre and category analysis
print("\n========== GENRE ANALYSIS ==========")
genre_df = (
    df[df["listed_in"] != "Unknown"]
    .assign(genre=lambda x: x["listed_in"].str.split(", "))
    .explode("genre")
)
top_genres = genre_df["genre"].value_counts().head(10)
print("\nTop 10 genres:")
print(top_genres)
plt.figure(figsize=(11, 6))
top_genres.sort_values().plot(kind="barh")
plt.title("Top 10 Netflix Genres")
plt.xlabel("Number of Titles")
plt.tight_layout()
plt.show()
directors_df = (
    df[df["director"] != "Unknown"]
    .assign(director=lambda x: x["director"].str.split(", "))
    .explode("director")
)
top_directors = directors_df["director"].value_counts().head(10)
print("\nTop 10 directors:")
print(top_directors)
plt.figure(figsize=(11, 6))
top_directors.sort_values().plot(kind="barh")
plt.title("Top 10 Directors on Netflix")
plt.xlabel("Number of Titles")
plt.tight_layout()
plt.show()
actors_df = (
    df[df["cast"] != "Unknown"]
    .assign(actor=lambda x: x["cast"].str.split(", "))
    .explode("actor")
)
top_actors = actors_df["actor"].value_counts().head(10)
print("\nTop 10 actors:")
print(top_actors)
plt.figure(figsize=(11, 6))
top_actors.sort_values().plot(kind="barh")
plt.title("Top 10 Frequently Listed Actors")
plt.xlabel("Number of Titles")
plt.tight_layout()
plt.show()

#241801041
#Darshan sp
# STEP 8: Word cloud of descriptions
print("\n========== DESCRIPTION ANALYSIS ==========")
descriptions = " ".join(
    df.loc[df["description"] != "Unknown", "description"]
)
wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="white",
    max_words=150
).generate(descriptions)
plt.figure(figsize=(14, 7))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Word Cloud of Netflix Descriptions")
plt.show()
```

```
#241801041
#Darshan sp
# STEP 9: Month in which most content was added
print("\n========== MONTHLY CONTENT ADDITIONS ==========")
monthly_counts = (
    df["month_added"]
    .dropna()
    .value_counts()
    .reindex([
        "January", "February", "March", "April",
        "May", "June", "July", "August",
        "September", "October", "November", "December"
    ], fill_value=0)
)
print(monthly_counts)
print("Most common month:", monthly_counts.idxmax())
plt.figure(figsize=(11, 5))
monthly_counts.plot(kind="bar")
plt.title("Netflix Titles Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

#241801041
#Darshan sp
# STEP 10: Countries producing Movies vs TV Shows
print("\n========== COUNTRY VS CONTENT TYPE ==========")
country_type = (
    countries.groupby(["country", "type"])
    .size()
    .unstack(fill_value=0)
)
country_type["Total"] = country_type.sum(axis=1)
country_type = country_type.sort_values(
    "Total", ascending=False
).head(10)
print(country_type)
country_type.drop(columns="Total").plot(
    kind="bar", figsize=(12, 6)
)
plt.title("Movies vs TV Shows by Top Countries")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

#241801041
#Darshan sp
# STEP 11: Oldest and newest titles by release year
print("\n========== OLDEST AND NEWEST TITLES ==========")
valid_years = df.dropna(subset=["release_year"]).copy()
oldest_titles = valid_years.nsmallest(10, "release_year")[
    ["title", "type", "release_year"]
]
newest_titles = valid_years.nlargest(10, "release_year")[
    ["title", "type", "release_year"]
]
print("\nOldest titles by release year:")
display(oldest_titles)
print("\nNewest titles by release year:")
display(newest_titles)

#241801041
#Darshan sp
# STEP 12: Genre co-occurrence heatmap
print("\n========== GENRE CO-OCCURRENCE HEATMAP ==========")
genre_lists = (
    df["listed_in"]
    .apply(lambda x: [] if x == "Unknown" else x.split(", "))
)
genre_frequency = Counter(
    genre for genres in genre_lists for genre in set(genres)
)
top_10_genres = [
    genre for genre, count in genre_frequency.most_common(10)
```

```
    genre for genre, count in genre_frequency.most_common(10)
]
genre_matrix = pd.DataFrame({
    genre: genre_lists.apply(lambda genres: int(genre in genres))
    for genre in top_10_genres
})
genre_corr = genre_matrix.corr()
plt.figure(figsize=(11, 8))
sns.heatmap(
    genre_corr,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    vmin=-1,
    vmax=1
)
plt.title("Genre Correlation Heatmap")
plt.tight_layout()
plt.show()

#241801041
#Darshan sp
# STEP 13: Final summary
print("\n========== FINAL SUMMARY ==========")
print("Total Netflix titles:", len(df))
print("Movies:", (df["type"] == "Movie").sum())
print("TV Shows:", (df["type"] == "TV Show").sum())
print("Distinct release years:", df["release_year"].nunique())
print("Distinct listed genres:", len(genre_frequency))
print("Top genre:", top_genres.index[0])
print("Top country:", top_countries.index[0])
print("Most common rating:", df["rating"].value_counts().index[0])
print("\nEXPERIMENT 6a COMPLETED SUCCESSFULLY!")
```

```
Dataset downloaded successfully!
Dataset shape: (7787, 12)

First five records:
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	TV Show	3%	NaN	João Miguel, Bianca Comparato, Michel Gomes, R...	Brazil	August 14, 2020	2020	TV-MA	4 Seasons	International TV Shows, TV Dramas, TV Sci-Fi &...	In a future where the elite inhabit an island ...
1	s2	Movie	7:19	Jorge Michel Grau	Demián Bichir, Héctor Bonilla, Oscar Serrano, ...	Mexico	December 23, 2016	2016	TV-MA	93 min	Dramas, International Movies	After a devastating earthquake hits Mexico Cit...
2	s3	Movie	23:59	Gilbert Chan	Tedd Chan, Stella Chung, Henley Hii, Lawrence ...	Singapore	December 20, 2018	2011	R	78 min	Horror Movies, International Movies	When an army recruit is found dead, his fellow...
3	s4	Movie	9	Shane Acker	Elijah Wood, John C. Reilly, Jennifer Connelly...	United States	November 16, 2017	2009	PG-13	80 min	Action & Adventure, Independent Movies, Sci-Fi...	In a postapocalyptic world, rag-doll robots hi...
4	s5	Movie	21	Robert Luketic	Jim Sturgess, Kevin Spacey, Kate Bosworth, Aar...	United States	January 1, 2020	2008	PG-13	123 min	Dramas	A brilliant group of students become card-coun...

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7787 entries, 0 to 7786
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       7787 non-null   object
 1   type          7787 non-null   object
 2   title         7787 non-null   object
 3   director      5398 non-null   object
 4   cast          7069 non-null   object
 5   country       7280 non-null   object
 6   date_added    7777 non-null   object
 7   release_year  7787 non-null   int64
 8   rating        7780 non-null   object
 9   duration      7787 non-null   object
 10  listed_in     7787 non-null   object
 11  description   7787 non-null   object
dtypes: int64(1), object(11)
memory usage: 730.2+ KB

Missing values:
show_id            0
type               0
title              0
director        2389
cast             718
country          507
date_added        10
release_year       0
rating             7
duration           0
listed_in          0
description        0
dtype: int64

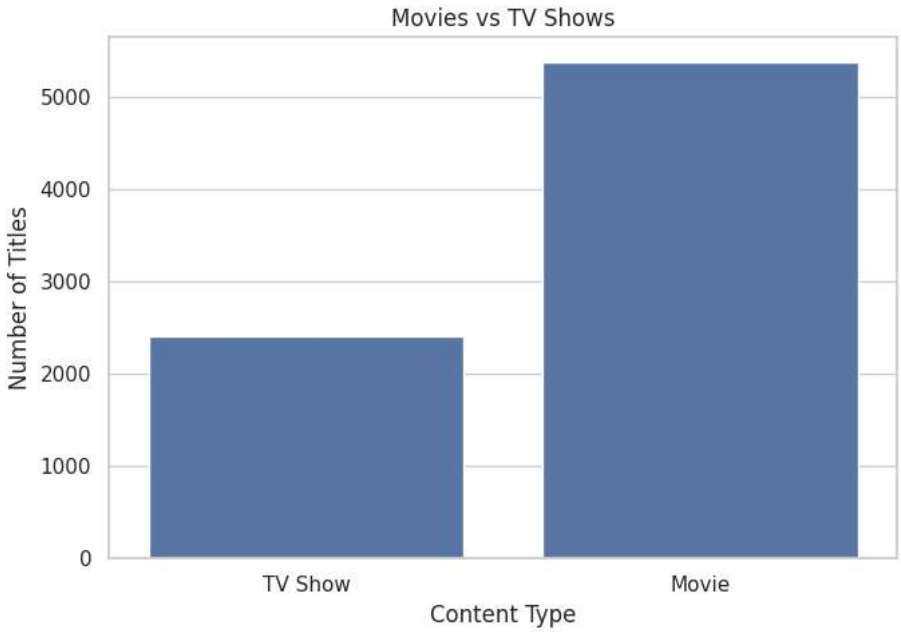
Cleaned dataset shape: (7787, 14)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
					João Miguel,						International	In a future

0	s1	TV Show	3%	Unknown	Bianca Comparato, Michel Gomes, R...	Brazil	2020-08-14	2020	TV-MA	4 Seasons	TV Shows, TV Dramas, TV Sci-Fi &...	...in a future where the elite inhabit an island ...
1	s2	Movie	7:19	Jorge Michel Grau	Demián Bichir, Héctor Bonilla, Oscar Serrano, ...	Mexico	2016-12-23	2016	TV-MA	93 min	Dramas, International Movies	After a devastating earthquake hits Mexico Cit...
2	s3	Movie	23:59	Gilbert Chan	Tedd Chan, Stella Chung, Henley Hii, Lawrence ...	Singapore	2018-12-20	2011	R	78 min	Horror Movies, International Movies	When an army recruit is found dead, his fellow...
3	s4	Movie	9	Shane Acker	Elijah Wood, John C. Reilly, Jennifer Connelly...	United States	2017-11-16	2009	PG-13	80 min	Action & Adventure, Independent Movies, Sci-Fi...	In a postapocalyptic world, rag-doll robots hi...
4	s5	Movie	21	Robert Luketic	Jim Sturgess, Kevin Spacey, Kate Bosworth, Aar...	United States	2020-01-01	2008	PG-13	123 min	Dramas	A brilliant group of students become card-coun...

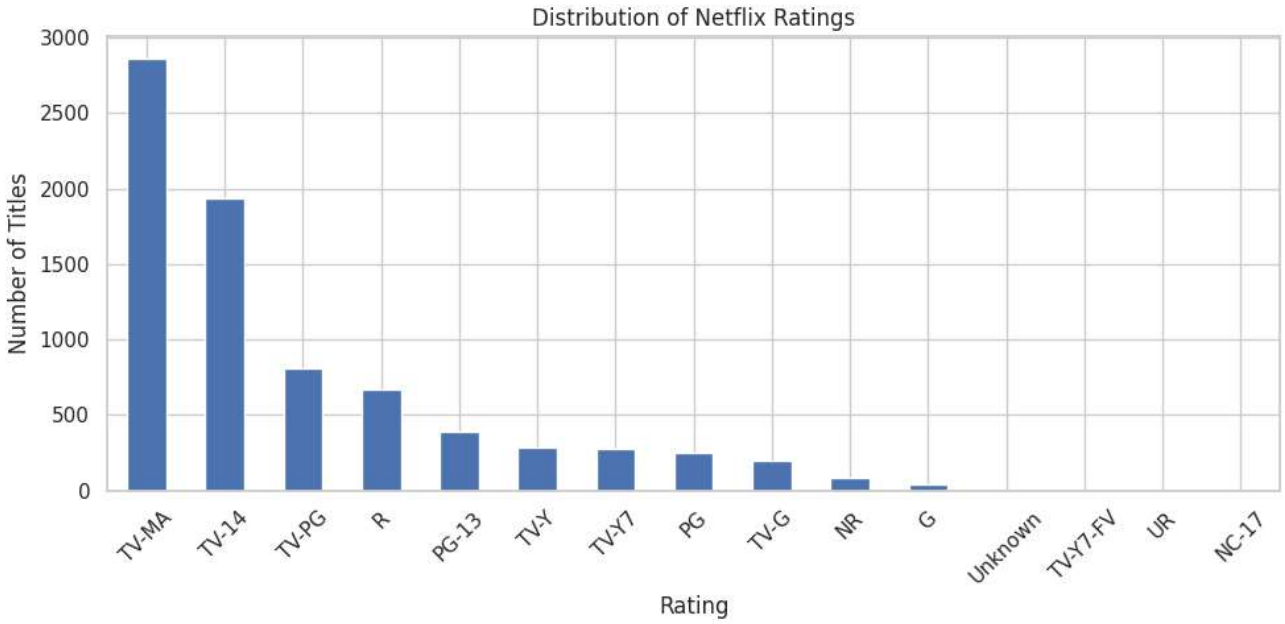
```
========== CONTENT DISTRIBUTION ==========

Movies and TV Shows:
type
Movie      5377
TV Show    2410
Name: count, dtype: int64
```

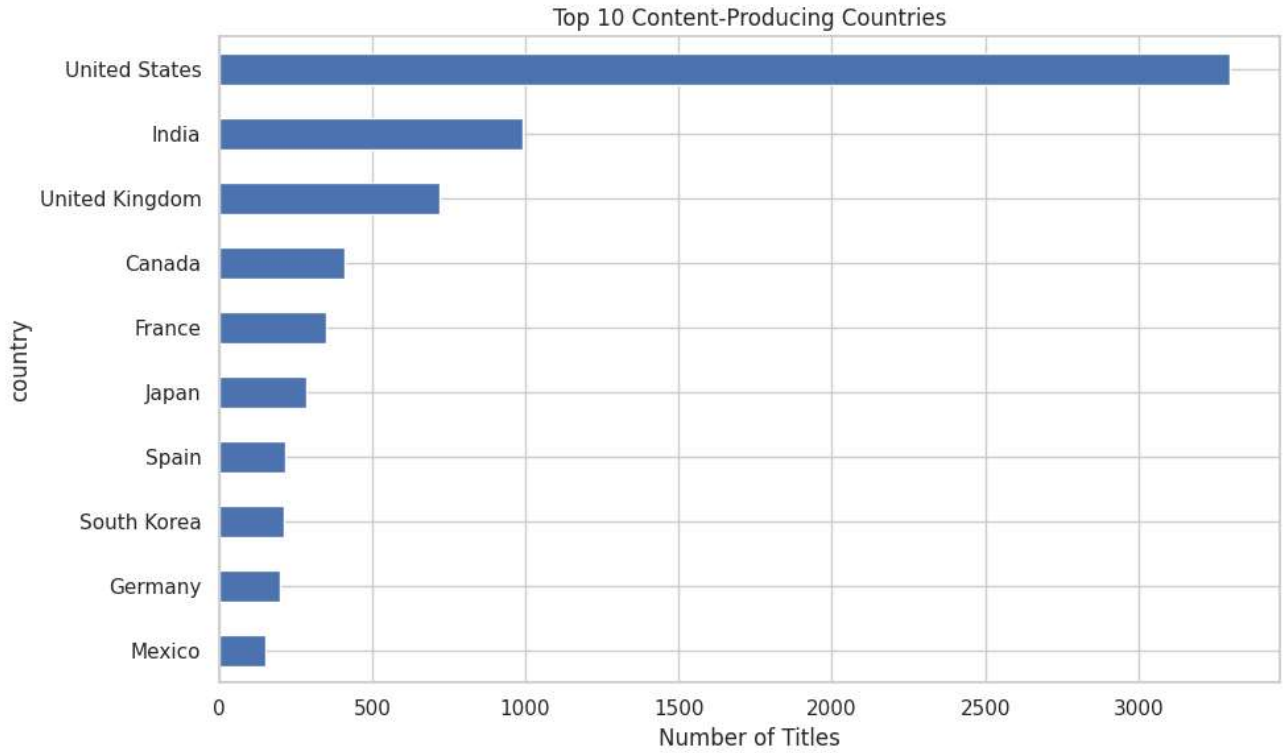



```
Rating distribution:
rating
TV-MA       2863
TV-14       1931
TV-PG        806
R            665
PG-13        386
TV-Y         280
TV-Y7        271
PG           247
TV-G         194
NR            84
G             39
Unknown        7
```

```
TV-Y7-FV        6
UR              5
NC-17           3
Name: count, dtype: int64
```

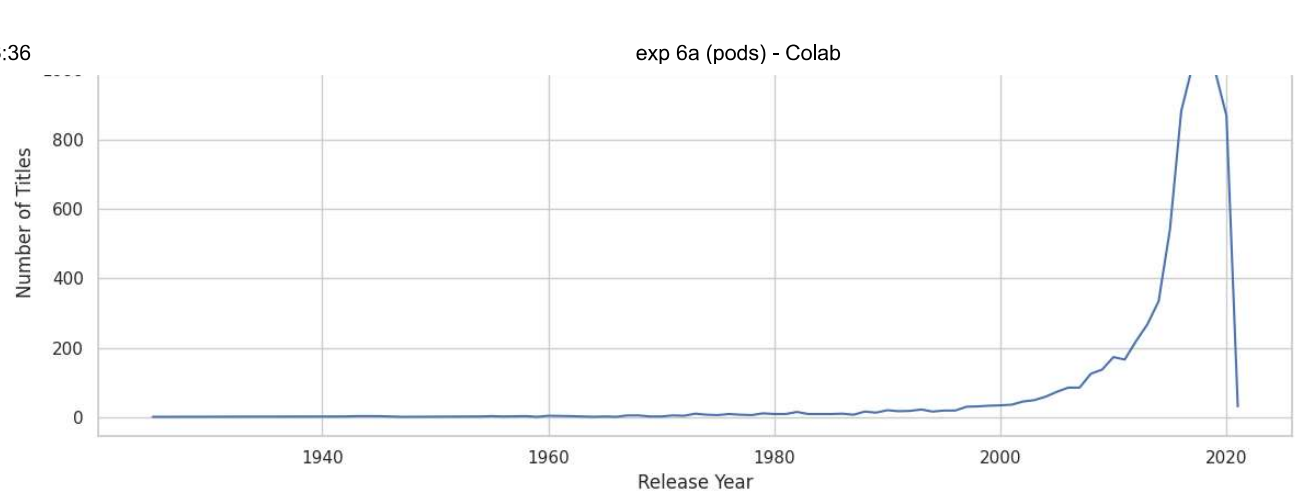


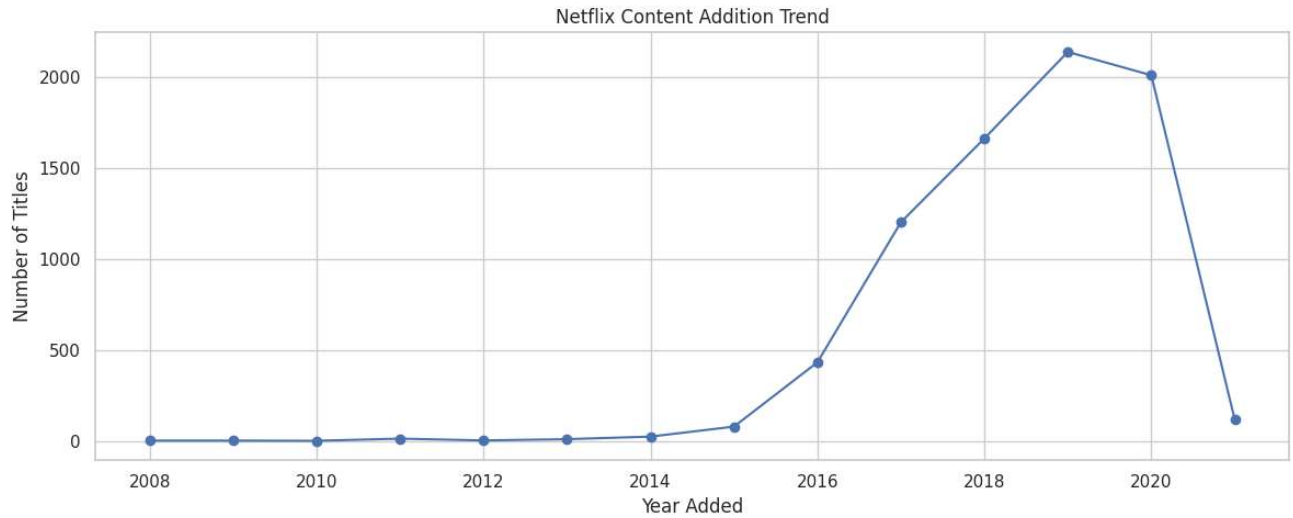

```
Top 10 countries:
country
United States     3296
India              990
United Kingdom     722
Canada             412
France             349
Japan              287
Spain              215
South Korea        212
Germany            199
Mexico             154
Name: count, dtype: int64
```




```
========== TIME-BASED TRENDS ==========
```








```
Titles released per year (latest 15 years):
release_year
2007      85
2008     125
2009     137
2010     173
2011     166
2012     219
2013     267
2014     334
2015     541
2016     882
2017    1012
2018    1121
2019     996
2020     868
2021      31
Name: count, dtype: int64

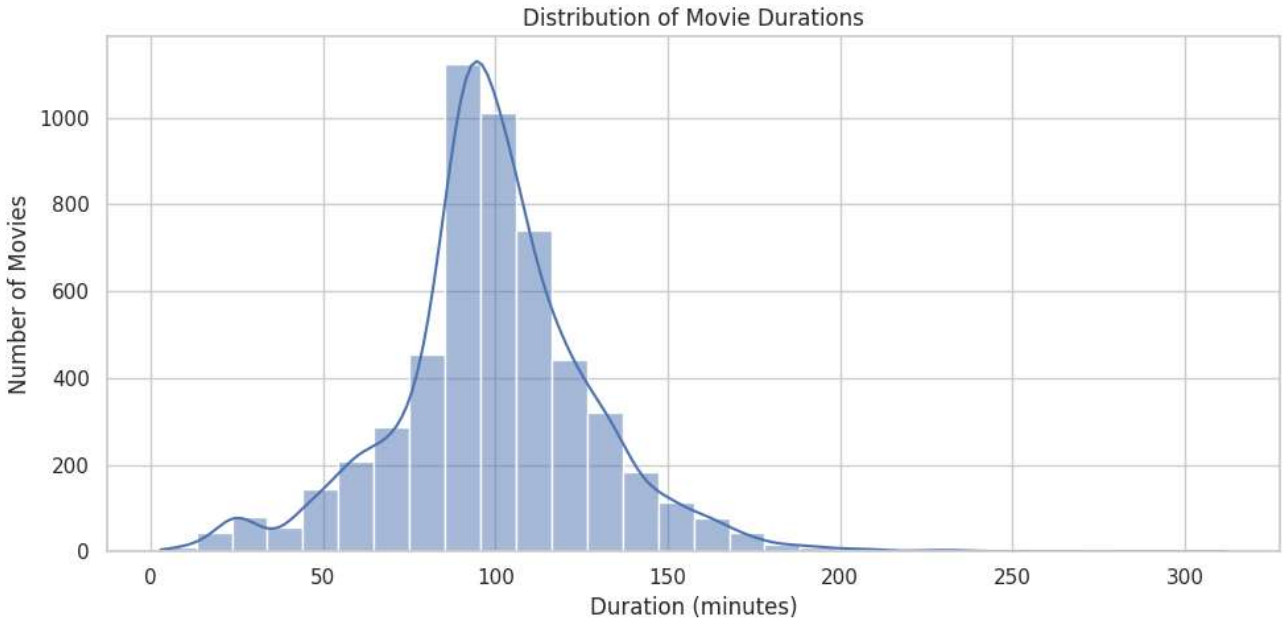
Titles added per year:
year_added
2008       2
2009       2
2010       1
2011      13
2012       3
2013      10
2014      24
2015      79
2016     432
2017    1201
2018    1661
2019    2136
2020    2008
2021     117
Name: count, dtype: int64

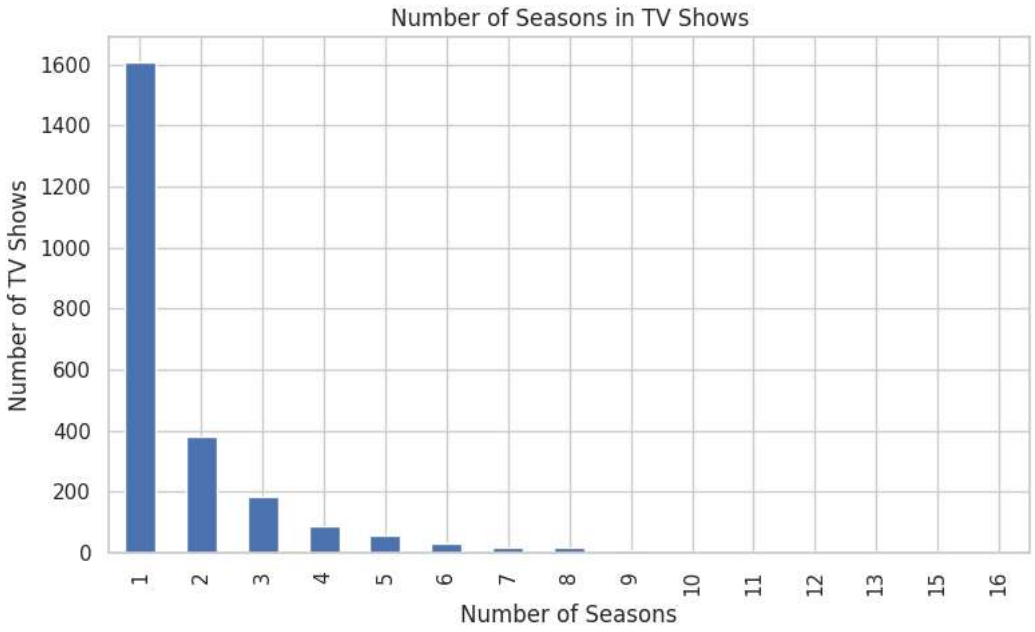
========== DURATION ANALYSIS ==========

Movie duration statistics in minutes:
count    5377.000000
mean       99.307978
std        28.530881
```

```
std         28.530881
min          3.000000
25%         86.000000
50%         98.000000
75%        114.000000
max        312.000000
Name: duration_value, dtype: float64

TV show season statistics:
count    2410.000000
mean        1.775934
std         1.596359
min         1.000000
25%         1.000000
50%         1.000000
75%         2.000000
max        16.000000
Name: duration_value, dtype: float64
```

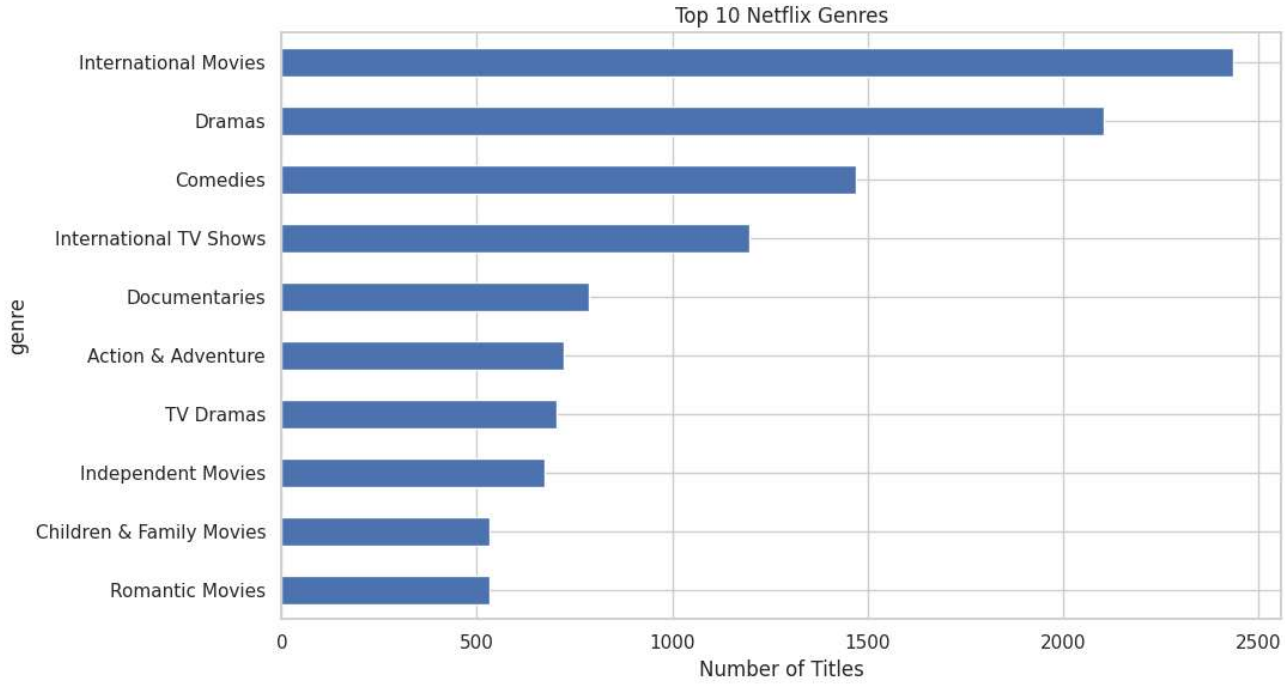




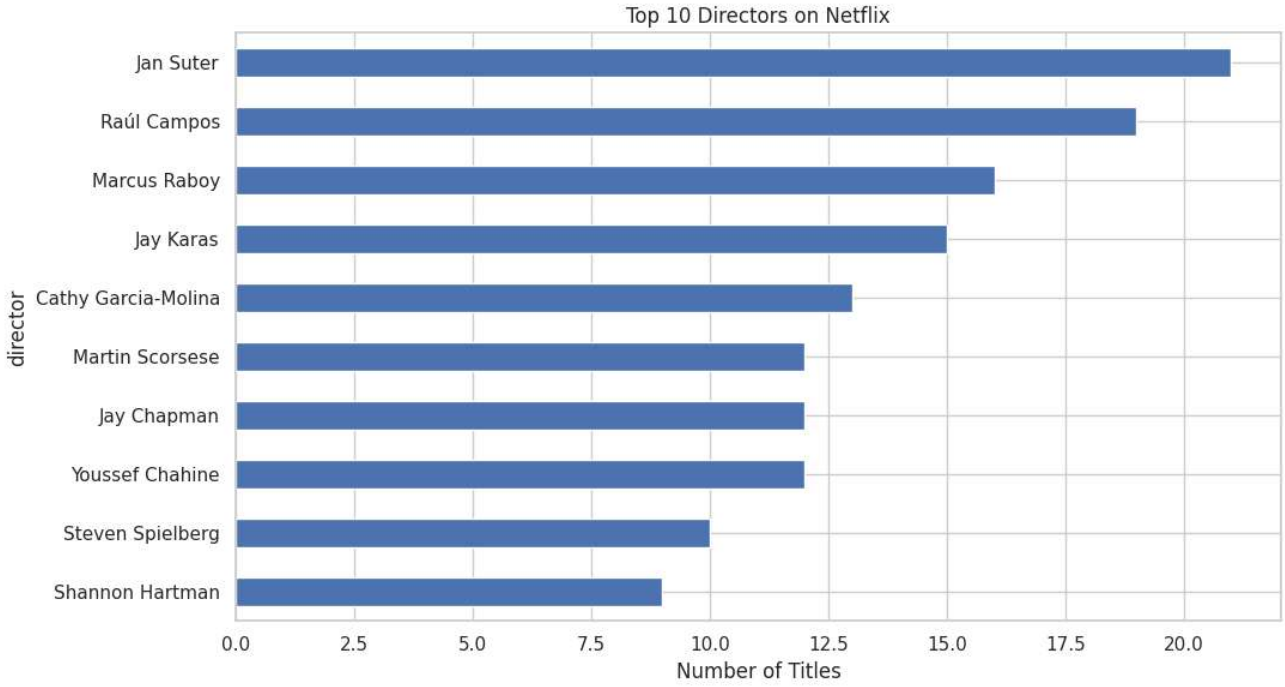

```
========== GENRE ANALYSIS ==========

Top 10 genres:
genre
International Movies        2437
Dramas                      2106
Comedies                    1471
International TV Shows      1199
Documentaries                786
Action & Adventure           721
TV Dramas                    704
```

```
Independent Movies           673
Children & Family Movies     532
Romantic Movies              531
Name: count, dtype: int64
```

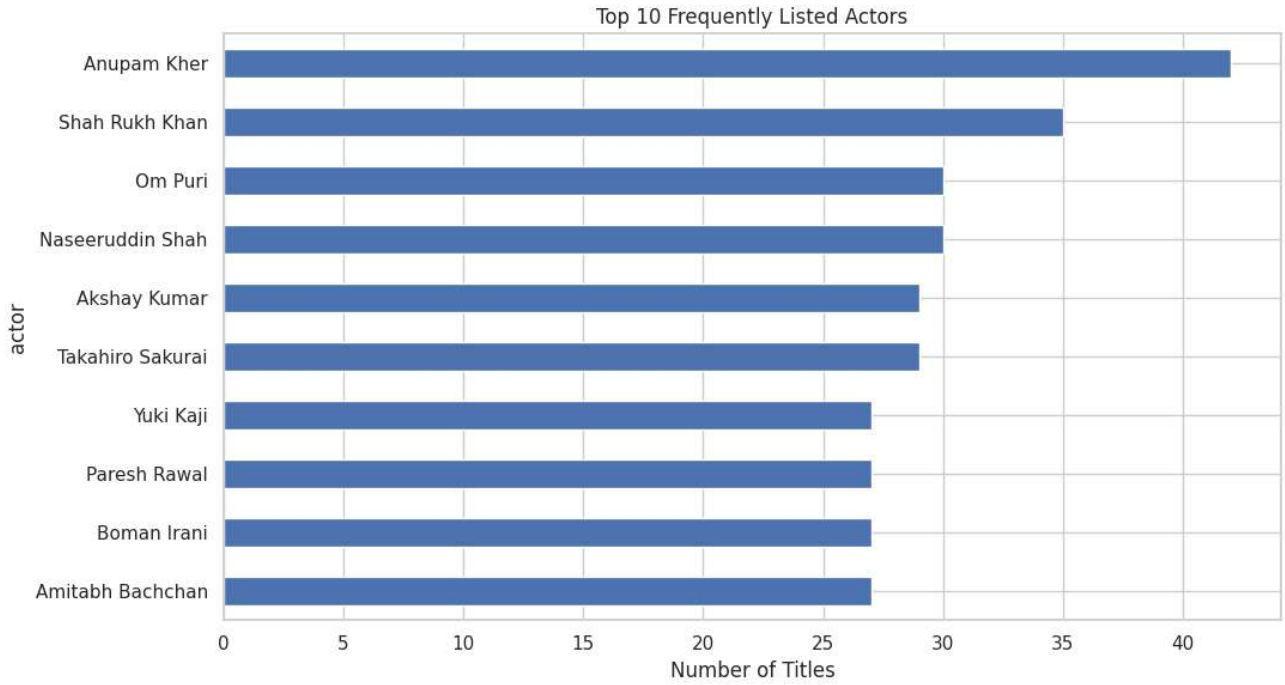



```
Top 10 directors:
director
Jan Suter              21
Raúl Campos            19
Marcus Raboy           16
Jay Karas              15
Cathy Garcia-Molina    13
Martin Scorsese        12
Youssef Chahine        12
Jay Chapman            12
Steven Spielberg       10
Shannon Hartman         9
Name: count, dtype: int64
```




```
Top 10 actors:
actor
Anupam Kher          42
Shah Rukh Khan       35
Naseeruddin Shah     30
```

```
Om Puri             30
Takahiro Sakurai    29
Akshay Kumar        29
Amitabh Bachchan    27
Boman Irani         27
Yuki Kaji           27
Paresh Rawal        27
Name: count, dtype: int64
```




```
========== DESCRIPTION ANALYSIS ==========
```




```
========== MONTHLY CONTENT ADDITIONS ==========
month_added
January      746
February     466
March        661
April        596
May          537
June         538
July         592
August       612
September    614
October      780
November     730
December     817
```

```
#241801041
#DARSHAN SP
#1: Import libraries and load dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error
import warnings

warnings.filterwarnings("ignore")

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"

df = pd.read_csv(url, parse_dates=["Month"], index_col="Month")
df.index.name = "Date"

print("Dataset loaded successfully!")
print(df.head())
print("\nDataset shape:", df.shape)
```

```
Dataset loaded successfully!
            Passengers
Date
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121

Dataset shape: (144, 1)
```

```
#241801041
#DARSHAN SP
# CELL 2: Explore the dataset

print("Dataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

# Time series plot
plt.figure(figsize=(12, 5))
plt.plot(df.index, df["Passengers"], marker="o")
plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.grid(True)
plt.show()

# Boxplot to identify possible outliers
plt.figure(figsize=(8, 4))
plt.boxplot(df["Passengers"])
```

```
plt.boxplot(df["Passengers"])
plt.title("Outlier Analysis of Airline Passengers")
plt.ylabel("Number of Passengers")
plt.show()

# Monthly seasonality
monthly_avg = df.groupby(df.index.month)["Passengers"].mean()

plt.figure(figsize=(10, 4))
plt.plot(monthly_avg.index, monthly_avg.values, marker="o")
plt.title("Average Passenger Count by Month")
plt.xlabel("Month")
plt.ylabel("Average Passengers")
plt.xticks(range(1, 13))
plt.grid(True)
plt.show()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
None

Statistical Summary:
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000
```

```
#241801041
#DARSHAN SP
# CELL 3: ADF test on original series

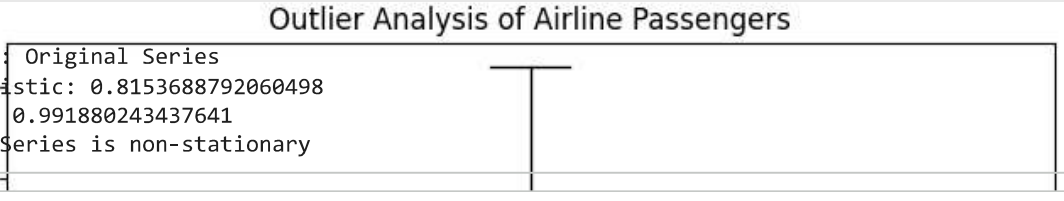
def adf_test(series, name):
    result = adfuller(series.dropna())

    print(f"\nADF Test: {name}")
    print("ADF Statistic:", result[0])
    print("p-value:", result[1])

    if result[1] <= 0.05:
        print("Result: Series is stationary")
    else:
        print("Result: Series is non-stationary")

    return result[1]

original_p = adf_test(df["Passengers"], "Original Series")
```




```
ADF Test: Original Series
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
Result: Series is non-stationary
```

```
#241801041
#DARSHAN SP
# CELL 4: Differencing and stationarity check

series = df["Passengers"].astype(float)
d = 0
stationary_series = series.copy()

p_value = adf_test(stationary_series, "Original Series")

while p_value > 0.05 and d < 2:
    d += 1
    stationary_series = series.diff(d) if d == 1 else series.diff().diff()
```

```
    p_value = adf_test(
        stationary_series,
        f"Series after {d} differencing"
    )

print("\nSelected ARIMA differencing order d =", d)

plt.figure(figsize=(12, 5))
plt.plot(stationary_series.index, stationary_series)
plt.title(f"Series After Differencing (d={d})")
plt.xlabel("Year")
plt.ylabel("Differenced Passengers")
plt.grid(True)
plt.show()
```

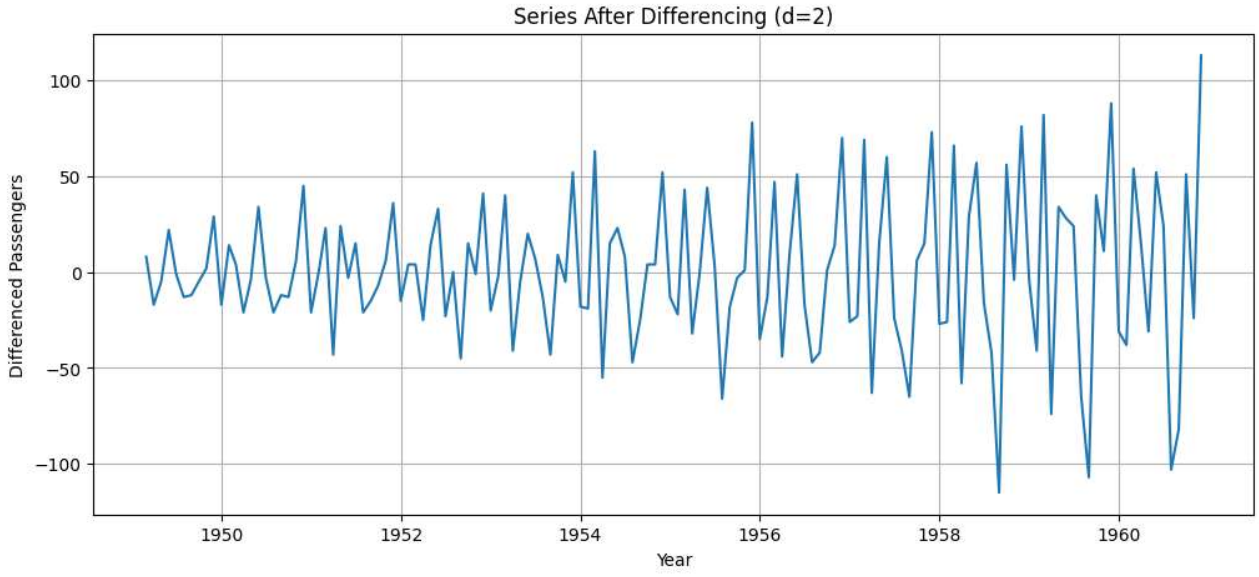



```
ADF Test: Original Series
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
Result: Series is non-stationary

ADF Test: Series after 1 differencing
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
Result: Series is non-stationary

ADF Test: Series after 2 differencing
ADF Statistic: -16.384231542468513
p-value: 2.7328918500142026e-29
Result: Series is stationary

Selected ARIMA differencing order d = 2
```




```
#241801041
#DARSHAN SP
# CELL 5: Train-test split and ARIMA model

split = int(len(df) * 0.8)

train = df["Passengers"].iloc[:split]
test = df["Passengers"].iloc[split:]
```

```
print("Training observations:", len(train))
print("Testing observations:", len(test))
print("ARIMA order:", (2, d, 1))

model = ARIMA(train, order=(2, d, 1))
model_fit = model.fit()

print("\nARIMA Model Summary:")
print(model_fit.summary())
```

```
Training observations: 115
Testing observations: 29
ARIMA order: (2, 2, 1)

ARIMA Model Summary:
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  115
Model:                 ARIMA(2, 2, 1)   Log Likelihood                -527.196
Date:                Mon, 12 Oct 2026   AIC                           1062.393
Time:                        11:23:07   BIC                           1073.302
Sample:                    01-01-1949   HQIC                          1066.820
                         - 07-01-1958
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.3396      0.092      3.705      0.000       0.160       0.519
ar.L2         -0.2108      0.105     -2.000      0.045      -0.417      -0.004
ma.L1         -0.9999     15.366     -0.065      0.948     -31.117      29.117
sigma2       634.0458   9732.652      0.065      0.948   -1.84e+04    1.97e+04
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                 3.60
Prob(Q):                              0.91   Prob(JB):                         0.17
Heteroskedasticity (H):               5.94   Skew:                             0.38
Prob(H) (two-sided):                  0.00   Kurtosis:                         2.58
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

```
#241801041
#DARSHAN SP
# CELL 6: Forecast and evaluation metrics

predictions = model_fit.forecast(steps=len(test))

predictions = pd.Series(
    np.asarray(predictions),
    index=test.index,
    name="Predicted"
)

mse = mean_squared_error(test, predictions)
mae = mean_absolute_error(test, predictions)
rmse = np.sqrt(mse)

print("Forecast Evaluation Metrics")
print("---------------------------")
print("MSE :", round(mse, 2))
print("MAE :", round(mae, 2))
print("RMSE:", round(rmse, 2))
```

```
results = pd.DataFrame({
    "Actual": test,
    "Predicted": predictions
})

print("\nActual vs Predicted Passengers:")
print(results)
```

```
Forecast Evaluation Metrics
---------------------------
MSE : 13064.54
MAE : 102.75
RMSE: 114.3

Actual vs Predicted Passengers:
            Actual   Predicted
Date
1958-08-01     505  497.654933
1958-09-01     404  490.923348
1958-10-01     359  490.046813
1958-11-01     310  493.980616
1958-12-01     337  498.313968
1959-01-01     360  501.769039
1959-02-01     342  504.841595
1959-03-01     406  507.969371
1959-04-01     396  511.196533
1959-05-01     420  514.445810
1959-06-01     472  517.681648
1959-07-01     548  520.908260
1959-08-01     559  524.134571
1959-09-01     463  527.362725
1959-10-01     407  530.591569
1959-11-01     362  533.820258
1959-12-01     405  537.048749
1960-01-01     417  540.277205
1960-02-01     391  543.505692
1960-03-01     419  546.734196
1960-04-01     461  549.962699
1960-05-01     472  553.191199
1960-06-01     535  556.419698
1960-07-01     622  559.648197
1960-08-01     606  562.876696
1960-09-01     508  566.105195
1960-10-01     461  569.333695
1960-11-01     390  572.562194
1960-12-01     432  575.790693
```

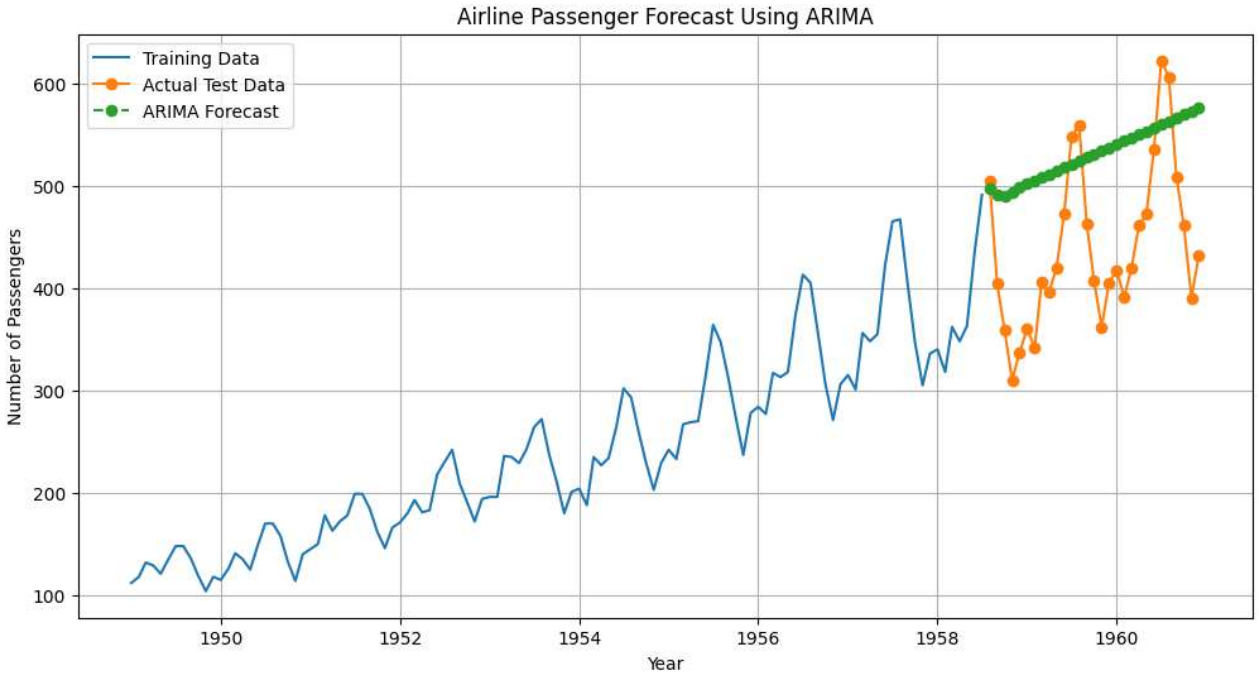
```
#241801041
#DARSHAN SP
# CELL 7: Plot actual values and forecasts

plt.figure(figsize=(12, 6))

plt.plot(train.index, train, label="Training Data")
plt.plot(test.index, test, label="Actual Test Data", marker="o")
plt.plot(
    predictions.index,
    predictions,
    label="ARIMA Forecast",
    linestyle="--",
    marker="o"
)
```

```
plt.title("Airline Passenger Forecast Using ARIMA")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.grid(True)
plt.show()

print("Experiment 6b completed successfully!")
```




```
Experiment 6b completed successfully!
```

```
#241801041
#DARSHAN SP
# CELL 8: Forecast next 12 months

final_model = ARIMA(df["Passengers"], order=(2, d, 1))
final_model_fit = final_model.fit()

future_forecast = final_model_fit.forecast(steps=12)

future_dates = pd.date_range(
    start=df.index[-1] + pd.offsets.MonthBegin(1),
    periods=12,
    freq="MS"
)

future_forecast = pd.Series(
    np.asarray(future_forecast),
    index=future_dates,
    name="Forecast"
)

print("Next 12 Months Passenger Forecast:")
print(future_forecast)

plt.figure(figsize=(12, 5))
```

```
plt.plot(df.index, df["Passengers"], label="Historical Data")
plt.plot(
    future_forecast.index,
    future_forecast,
    label="Future Forecast",
    marker="o",
    linestyle="--"
)
plt.title("Airline Passenger Forecast for Next 12 Months")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.grid(True)
plt.show()
```

```
Next 12 Months Passenger Forecast:
1961-01-01    466.218395
1961-02-01    471.918238
1961-03-01    468.409366
1961-04-01    467.801585
1961-05-01    470.389599
1961-06-01    473.551184
1961-07-01    476.211413
1961-08-01    478.549284
1961-09-01    480.876449
1961-10-01    483.272316
1961-11-01    485.697021
1961-12-01    488.117295
```

```
#241801041
#Darshan SP
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
url = "https://raw.githubusercontent.com/tanishq21/Mall-Customers/main/M
df = pd.read_csv(url)
print("Dataset loaded successfully!")
print("Shape:", df.shape)
display(df.head())
```

```
Dataset loaded successfully!
Shape: (200, 5)
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
#241801041
#Darshan SP
print("First five records:")
display(df.head())
print("Last five records:")
display(df.tail())
print("Dataset shape:", df.shape)
print("\nData types:")
print(df.dtypes)
print("\nMissing values:")
print(df.isnull().sum())
print("\nDuplicate records:", df.duplicated().sum())
df = df.dropna().drop_duplicates().reset_index(drop=True)
print("\nCleaned dataset shape:", df.shape)
print("\nSummary statistics:")
display(df.describe(include="all"))
```

First five records:

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Last five records:

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	35	120	79

```
#241801041
#Darshan SP
features = ["Annual Income (k$)", "Spending Score (1-100)"]
X = df[features].copy()
print("Selected features:")
display(X.head())
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Standardization completed!")
print("Standardized feature shape:", X_scaled.shape)
plt.figure(figsize=(8, 5))
plt.scatter(X[features[0]], X[features[1]])
plt.title("Annual Income vs Spending Score")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.grid(True)
plt.show()
```

```
CustomerID              0
Genre                   0
Age                     0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64

Duplicate records: 0

Cleaned dataset shape: (200, 5)

Summary statistics:
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200	200.000000	200.000000	200.000000
unique	NaN	2	NaN	NaN	NaN
top	NaN	Female	NaN	NaN	NaN
freq	NaN	112	NaN	NaN	NaN

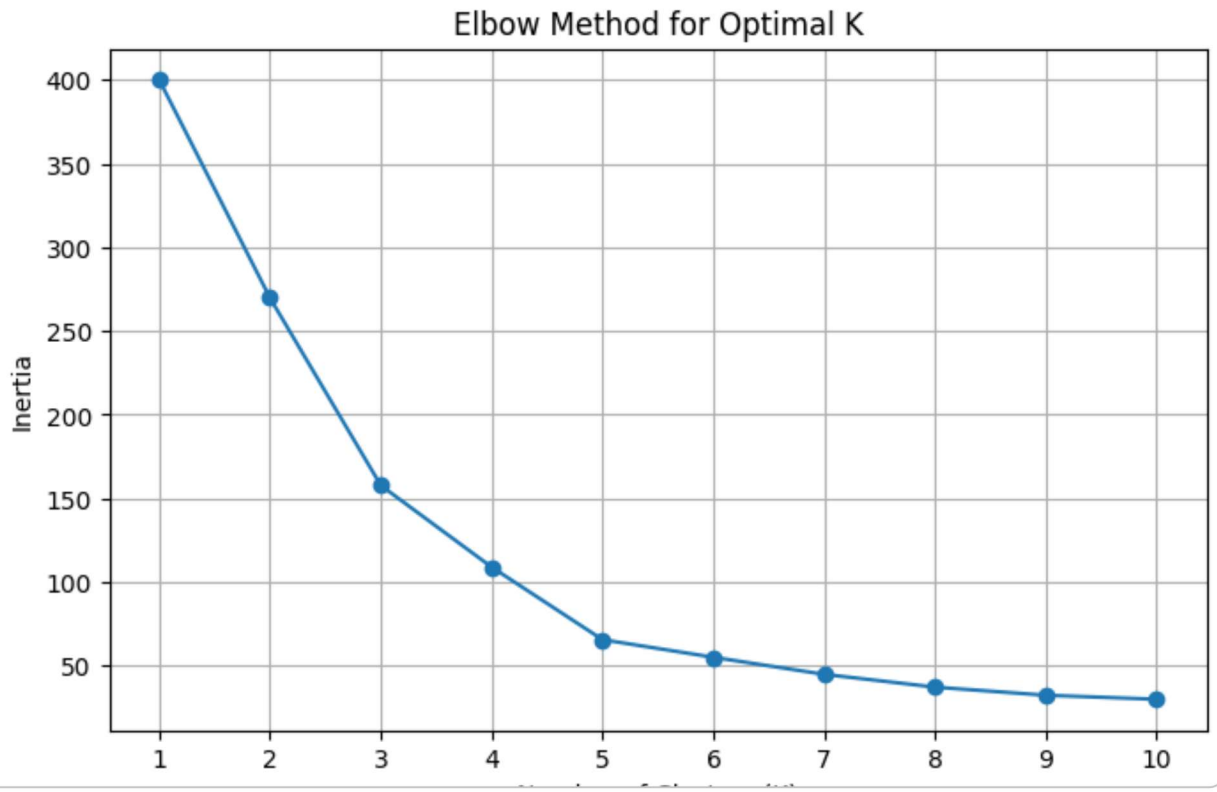
```
Selected features:
```

	Annual Income (k$)	Spending Score (1-100)
mean	100.500000	38.850000
std	NaN	NaN
min	1.000000	1.000000
25%	50.750000	28.750000
50%	100.500000	36.000000
75%	150.250000	49.000000
max	200.000000	137.000000

```
Standardization completed!
Standardized feature shape: (200, 2)
```




```
#241801041
#Darshan SP
inertia = []
for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertia.append(model.inertia_)
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), inertia, marker="o")
plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()
```




```
#241801041
#Darshan SP
optimal_k = 5
kmeans = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)
df["Cluster"] = kmeans.fit_predict(X_scaled)
print("K-Means clustering completed!")
print("\nCustomer cluster assignments:")
display(df.head(10))
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
centroid_df = pd.DataFrame(centroids, columns=features)
centroid_df.index.name = "Cluster"
print("\nCluster centroid coordinates:")
display(centroid_df)
```

```
K-Means clustering completed!

Customer cluster assignments:
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4
5	6	Female	22	17	76	2
6	7	Female	35	18	6	4
7	8	Female	23	18	94	2
8	9	Male	64	19	3	4
9	10	Female	30	19	72	2

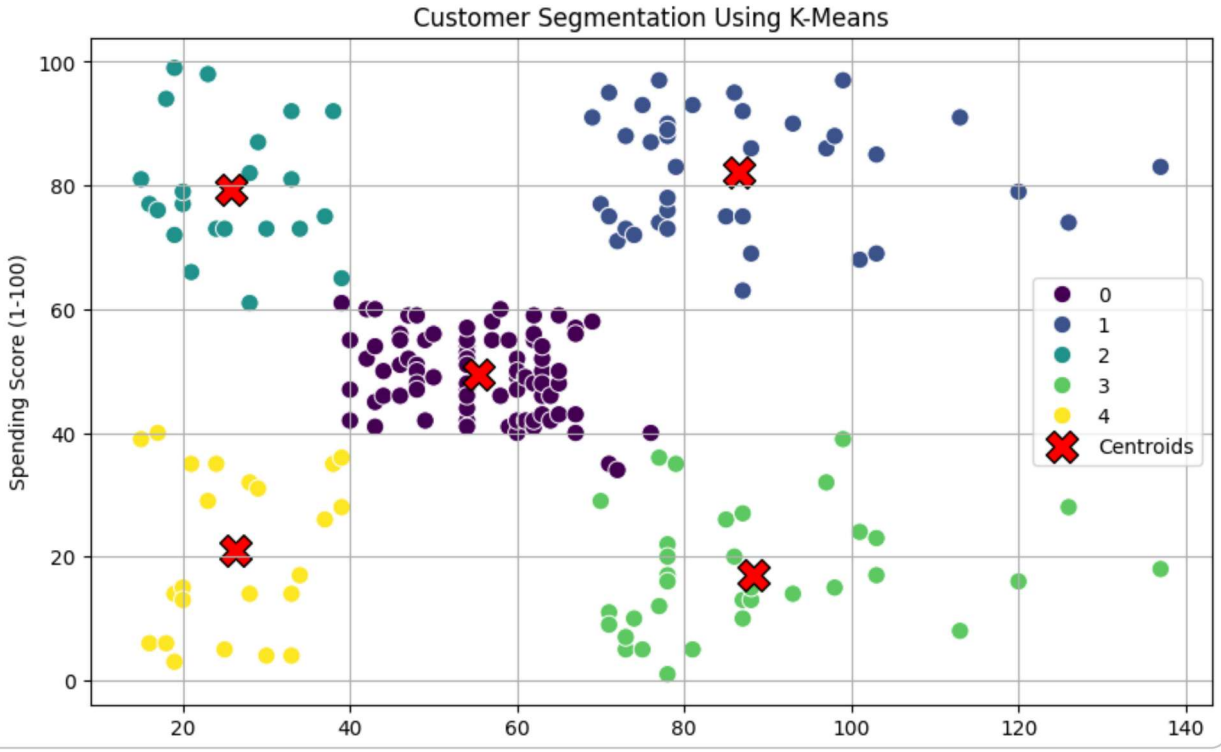
```
Cluster centroid coordinates:
```

Cluster	Annual Income (k$)	Spending Score (1-100)
0	55.296296	49.518519
1	86.538462	82.128205
2	25.727273	79.363636
3	88.200000	17.114286
4	26.304348	20.913043

Next steps: Generate code with `centroid_df` New interactive sheet

```
#241801041
#Darshan SP
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x="Annual Income (k$)",
    y="Spending Score (1-100)",
    hue="Cluster",
    palette="viridis",
    s=80
)
plt.scatter(
```

```
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=250,
    c="red",
    edgecolors="black",
    label="Centroids"
)
plt.title("Customer Segmentation Using K-Means")
plt.legend()
plt.grid(True)
plt.show()
```




```
#241801041
#Darshan SP
print("Customers in each cluster:")
print(df["Cluster"].value_counts().sort_index())
cluster_summary = df.groupby("Cluster").agg(
    Customer_Count=("Cluster", "size"),
    Average_Annual_Income=("Annual Income (k$)", "mean"),
    Average_Spending_Score=("Spending Score (1-100)", "mean")
).round(2)
print("\nCluster Analysis:")
display(cluster_summary)
```

```
Customers in each cluster:
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64

Cluster Analysis:
```

Cluster	Customer_Count	Average_Annual_Income	Average_Spending_Score
0	81	55.30	49.52
1	39	86.54	82.13
2	22	25.73	79.36
3	35	88.20	17.11

Next steps: Generate code with `cluster_summary` New interactive sheet